大连理工大学本科毕业论文（设计）

基于大模型的软件开发多智能体协作方法研究

Research on Multi-Agent Collaboration Methods for Software Development Based on Large Models

学　　院：软件学院

专　　业：软件工程

学 生 姓 名：徐远达

学　　号：20222241093

指 导 教 师：刘 晗

评 阅 教 师：张晓彤

完 成 日 期：2026 年 5 月 30 日

大连理工大学

Dalian University of Technolo

原创性声明

本人郑重声明：本人所呈交的毕业论文（设计），是在指导老师的指导下独立进行研究所取得的成果。毕业论文（设计）中凡引用他人已经发表或未发表的成果、数据、观点等，均已明确注明出处。除文中已经注明引用的内容外，不包含任何其他个人或集体已经发表或撰写过的科研成果。对本文的研究成果做出重要贡献的个人和集体，均已在文中以明确方式标明。

本声明的法律责任由本人承担。

作者签名：　　日　期：2026 年 5 月 30 日

关于使用授权的声明

本人在指导老师指导下所完成的毕业论文（设计）及相关的资料（包括图纸、试验记录、原始数据、实物照片、图片、录音带、设计手稿等），知识产权归属大连理工大学。本人完全了解大连理工大学有关保存、使用毕业论文（设计）的规定，本人授权大连理工大学可以将本毕业论文（设计）的全部或部分内容编入有关数据库进行检索，可以采用任何复制手段保存和汇编本毕业论文（设计）。如果发表相关成果，一定征得指导教师同意，且第一署名单位为大连理工大学。本人离校后使用毕业论文（设计）或与该论文直接相关的学术论文或成果时，第一署名单位仍然为大连理工大学。

论文作者签名：　　日　期：2026 年 5 月 30 日

指导老师签名：　　日　期：2026 年 5 月 30 日

摘要

传统软件开发模式存在成本高、周期长、编码自动化程度低等问题，随着大语言模型的快速发展，多智能体协作成为实现自动化软件开发的重要方向。现有的基于大语言模型的多智能体软件开发框架虽然对多智能体软件开发做出了有益探索，但是它们在协作过程上仍然存在智能体角色冗余、软件开发过程性反馈缺失等缺陷，导致自动化软件开发的最终成果在可用性上存在明显不足。

为此，本文提出一种分层协作与平等对话协作互相融合的多智能体软件开发协作方法，旨在提升智能体团队开发软件的可用性与可靠性。本文设计了包含“规划师-开发者-评估者”在内的多智能体软件开发团队，构建分步骤软件开发工作流。通过需求分级拆解、验收标准协商、自测与复现闭环，添加了密集的软件开发过程性反馈，实现了多智能体在需求分析、代码开发、质量验证间的全流程协同。此外，通过提供沙箱隔离环境，为多智能体自动化软件开发提供了必要的安全性保障。

本文选取 ChatDev、MetaGPT 作为基线，在 SoftwareDev 数据集子集上开展对比实验，统计耗时、词元消耗、代码规模及软件可用性等评估指标。实验结果表明，本文所提方法在软件成果可用性上达到 6/7，较 ChatDev（3/7）提升 42.8%、较 MetaGPT（4/7）提升 28.5%，尤其在较复杂的游戏类软件项目开发中优势显著。

关键词：大语言模型；多智能体；代码自动生成

Research on Multi-Agent Collaboration Methods for Software Development Based on Large Models

Abstract

Traditional software development suffer from high costs, long cycles and low levels of coding automation. With the rapid advancement of large language models, multi-agent collaboration has emerged as a critical approach to automated software development. Although existing LLM-based multi-agent software development frameworks have made valuable explorations in this field, they still suffer from redundant agent roles and insufficient procedural feedback during collaboration, resulting in prominent deficiencies in the usability of final automated development outputs.

To address these issues, this thesis proposes a multi-agent software development collaboration method integrating hierarchical collaboration and equal dialogue collaboration, aiming to improve the usability and reliability of software developed by agent teams. A multi-agent team consisting of planner, developer and evaluator is designed, and a stepwise software development workflow is established. Dense procedural feedback is introduced through hierarchical requirement decomposition, acceptance criteria negotiation, self-testing and reproduction closed-loop management, enabling full-process coordination of multi-agents across requirement analysis, coding development and quality verification. Furthermore, a sandbox isolation environment is deployed to ensure essential security for automated multi-agent software development.

ChatDev and MetaGPT are selected as baseline methods for comparative experiments on subsets of the SoftwareDev dataset. Evaluation metrics including development time, token consumption, code scale and software usability are analyzed. Experimental results demonstrate that the proposed method achieves a software usability score of 6/7, representing a 42.8% improvement over ChatDev (3/7) and a 28.5% increase compared with MetaGPT (4/7). The method exhibits particularly outstanding advantages in developing complex game software projects.

Key Words：LLM；Multi-Agent；Automatic Code Generation

目录

摘要 I
Abstract II
1 文献综述 1
1.1 研究背景和意义 1
1.2 国内外研究现状分析 1
1.3 现有研究存在的问题与不足 3
1.3 本文结构 4
1.4 本文主要工作 4
1.5 本章小结 5
2 相关技术介绍 6
2.1 大语言模型 6
2.2 智能体技术 6
2.3 多智能体系统 7
2.4 智能体实现框架 LangChain 10
2.5 本章小结 12
3 软件开发多智能体协作方法 13
3.1 多智能体团队概述 13
3.2 智能体能力获取和增强 13
3.2.1 多样化工具 13
3.2.2 技能系统 15
3.3 多智能体团队协作 16
3.3.1 开发工作流 16
3.3.2 任务分解 17
3.4 沙盒运行环境 18
3.5 实现细节 20
3.5.1 通信工具 20
3.5.2 消息传递的具体流程和数据格式 21
3.5.3 专家智能体的独立状态持久化 21
3.5.4 智能体状态管理 22
3.5.5 棕地开发支持 24
3.6 本章小结 25

4 实验设计与结果分析 26
4.1 基线 26
4.2 数据集与评估指标 26
4.3 对比实验 26
4.4 角色消融实验 32
4.5 协作过程分析 33
4.6 不足与展望 38
结论 40
参考文献 41
附录 A Software-7 软件开发任务 44
附录 B 软件成果物功能人工验证详情、AgentTeam 软件成果物 45
1 ChatDev 软件成果物功能人工验证详情 45
1.1 验证概述 45
1.2 贪吃蛇游戏（ChatDev-0）验证详情 45
1.3 打砖块游戏（ChatDev-1）验证详情 45
1.4 2048 游戏（ChatDev-2）验证详情 46
1.5 飞翔的小鸟游戏（ChatDev-3）验证详情 46
1.6 坦克大战游戏（ChatDev-4）验证详情 47
1.7 Excel 数据处理工具（ChatDev-5）验证详情 48
1.8 客户管理系统（ChatDev-6）验证详情 48
2 MetaGPT 软件成果物功能人工验证详情 48
2.1 验证概述 48
2.2 贪吃蛇游戏（MetaGPT-0）验证详情 49
2.3 打砖块游戏（MetaGPT-1）验证详情 49
2.4 2048 游戏（MetaGPT-2）验证详情 50
2.5 飞翔的小鸟游戏（MetaGPT-3）验证详情 50
2.6 坦克大战游戏（MetaGPT-4）验证详情 50
2.7 Excel 数据处理工具（MetaGPT-5）验证详情 51
2.8 客户管理系统（MetaGPT-6）验证详情 51
3 AgentTeam 软件功能成果物功能人工验证详情 51
3.1 验证概述 51
3.2 贪吃蛇游戏（AgentTeam-0）验证详情 52

3.3 打砖块游戏（AgentTeam-1）验证详情 52
3.4 2048 游戏（AgentTeam-2）验证详情 53
3.5 飞翔的小鸟游戏（AgentTeam-3）验证详情 53
3.6 坦克大战游戏（AgentTeam-4）验证详情 53
3.7 Excel 数据处理工具（AgentTeam-5）验证详情 54
3.8 客户管理系统（AgentTeam-6）验证详情 54
4 AgentTeam 软件成果物 55
4.1 贪吃蛇游戏（AgentTeam-0） 55
4.2 打砖块游戏（AgentTeam-1） 56
4.3 2048 游戏（AgentTeam-2） 56
4.4 飞翔的小鸟游戏（AgentTeam-3） 57
4.5 坦克大战游戏（AgentTeam-4） 57
4.6 Excel 数据处理工具（AgentTeam-5） 58
4.7 客户管理系统（AgentTeam-6） 58
修改记录 60
致谢 61

1 文献综述

1.1 研究背景和意义

随着大语言模型的语义理解能力、推理能力不断增强，人们将基于大语言模型的智能体视作实现通用人工智能的有前景的途径。以大语言模型为大脑，通过提示词工程和外部工具调用，可以打造出擅长特定任务的智能体，但是单个智能体的能力毕竟有限，尤其是在面临复杂任务时，比如需要应对复杂需求的软件开发或代码生成。人类社会特别擅长通过团队合作、专门化实现共同目标，多智能体系统充分汲取集体智慧，尝试利用不同角色的智能体的长处和独特观点，解决复杂问题[1]。

在软件开发向复杂化、智能化转型的背景下，多智能体系统（MAS）凭借分布式协作、自主决策、动态适应的核心特性，为软件开发提供了全新技术路径，应用价值显著。其可将复杂开发任务拆解为子任务，分配给不同智能体自主执行，通过智能通信实现需求分析、代码编写、测试验证全流程协同，有效解决传统开发中协同成本高、周期长的问题，大幅提升开发效率。同时，智能体具备异常检测和自我调整能力，可实时排查代码漏洞、弥补测试疏漏，减少人为失误，保障软件质量与可靠性。此外，其分布式架构和自学习能力，能让软件快速适配需求变化和环境升级，无需大规模重构，提升系统可扩展性、降低维护成本，还可降低软件开发技术门槛，推动开发向自动化、智能化转型，为复杂软件项目提供高效可靠的技术支撑，具有重要的理论与实际应用价值。

1.2 国内外研究现状分析

早期使用大语言模型进行代码生成所使用的方法多是微调预训练模型[2–5]或者简单地将大语言模型与工具接口结合为单个智能体以完成代码生成或软件开发任务，比如工具增强的代码生成[6,7]，基于规划的代码生成[8]，融入自我改进机制的单智能体[9–11]。

在面对复杂需求时，单智能体软件开发解决方案常常难以奏效，同时其在决策水平、并行任务处理能力等方面存在缺陷，因此多智能体软件开发系统逐渐兴起[12,13]。

Self-collaboration Code Generation via ChatGPT 首次提出将软件开发的瀑布模型方法学引入 LLMs 的协作，发明了一个使用 LLM 进行代码生成的自协作框架[14]。在这个框架中，多个 LLM 作为软件开发特定阶段的不同“专家”。他们根据指定的指令进行协作和互动，形成一个虚拟团队，促进彼此的工作。最终，虚拟团队在不需要人为干预的情况下协作完成代码生成任务。该文章揭示了角色分工、彼此交互对于提高质量的重要作用，同时显示了多智能体系统用于代码生成的巨大潜力；但是各个智能体的协作是基于保存在 BlackBoard 中先决阶段软件成果物品，单个智能体并没有自己的协作记忆；

所用数据集多是函数级别的开发任务，尽管有较复杂的案例研究，该框架研究的任务难度距离真实软件开发场景仍有较大差距。

CAMEL（对话式智能体，探索大语言模型社会的思维模式）旨在借助一个能够自主交流的用户智能体，代替人类与聊天模型解决任务，该用户智能体能够在没有人类干预的情况下引导对话朝着解决任务的方向进行[15]。CAMEL 创新性地提出了 Inception Prompting Method（启始式提示 / 植入式提示），主要由任务细化者系统提示词 PT，助手系统提示词 PA, 以及用户系统提示词 PU 三部分组成，其中 PT 是用于将简短模糊的用户需求进行创意性的扩展丰富，PA 和 PU 具有一定的对称性，都包含涵被分配的任务和角色、可访问的外部工具、通信协议、终止条件以及避免不良行为的约束或要求。CAMEL 框架面临智能体角色翻转、对话信息超载、聊天模型敷衍式回复等问题。

ChatDev: Communicative Agents for Software Development 借鉴瀑布模型，提出了 Chat-Chain 工作流，用以准确表述软件开发的各个环节，并利用 Inception 提示机制催眠初始化 Instructor 和 Assistant 两个智能体，使它们通过多轮对话完成每个子任务[16]。该工作流还明确定义了跨越阶段的长期记忆和用于维持单阶段内对话连续性、辅助上下文感知决策的短期记忆机制。ChatDev 提出了 Communicative Dehallucination 机制，在子任务解决过程中，鼓励 Assistant 通过主动反问 Instructor 来获得更加详细的建议，有效地缓解了某些过于模糊宽泛的指令常常导致的编码幻觉问题。但是 ChatDev 仍然存在以下不足：该框架下智能体对清晰详细的需求非常依赖；虽然强调了软件的完整性、可执行性、一致性和整体质量，但未考虑其他因素，如功能、鲁棒性、安全性和用户友好性；多智能体系统需要更多的 Token 和时间，增加了计算需求和环境影响；并没有可以跨越多个软件开发任务的经验库存和检索机制，在面对未见开发任务时无法有效利用已有的成功经验提高效率；虚拟开发团队和 Chat-Chain 可以通过配置文件人为定制，但仍然耗时耗力。

MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework 为了应对简单串联起多个智能体而导致的级联幻觉，选择将软件开发领域的标准化操作流程编码到提示词序列中来简化改善工作流[17]。此外，MetaGPT 提出了全局共享消息池以及消息订阅机制，确保了有效信息在智能体间的高效传输。MetaGPT 还引入了具备执行反馈的迭代式编程机制，克服了依赖 LLM 进行静态代码审阅中出现的幻觉问题。MetaGPT 为较复杂的软件开发任务设计了一些新的评估指标，比如生产效率（定义为每行代码消耗的 Token 数），为全面评估软件开发多智能体系统提供了新视角。但是该框架生成的代码在可执行性仍然存在缺陷，这是由于智能体在每步单纯遵守人类预定义的静态指令、优化规则，降低了灵活性和适应性。

除了 Self-Collaboration、ChatDev、MetaGPT 这类专门用于代码生成或者软件开发的多智能体协作框架之外，还有一些通用的多智能体协作框架，比如 AgentVerse、MACNet，它们被设计用来处理逻辑推理、函数级别代码生成、软件开发、通用问答、文章撰写等任务。

为了增强自治多智能体实现目标的有效性，AgentVerse 模拟人类群体的问题解决过程，将问题解决分为四个关键阶段：专家招募、协同决策、行动执行和评估[18]。AgentVerse 将整个过程建模为马尔可夫决策过程（MDP），其特征为元组（S，A，T，R，G）。这包括自主的智能体和环境状态空间 S、解决方案和动作空间 A、转移函数 T：S × A → S、奖励函数 R 和目标空间 G。

MACNet 是一个用于规模化扩展多智能体协作的多智能体框架，其提供多种预定义的多智能体协作拓扑模板结构[19]。类似于 ChatDev，MACNet 同样将多智能体交互建模为多次双智能体间的多轮对话，区别之处在于同一个任务输入，在经历带有分支结构的拓扑模板时，过程中会产生多种解决方案。解决方案的多样性来自于拓扑结构中每个节点上的智能体的系统提示词的多样性和不同层拓扑排序深度决定的模型温度，以及某些节点上的智能体产出的解决方案本身就是对接收到的多个不同的解决方案的按照一定策略有机融合后的产物。

1.3 现有研究存在的问题与不足

目前的多智能体软件开发框架，无论是以 ChatDev 为代表的多阶段双智能体对话驱动的框架，还是还是以 MetaGPT 为代表的标准化流程驱动的框架，各自都存在一些问题和不足，主要不足在于智能体工具配备不足、多智能体协作缺陷和人类干预支持薄弱。

智能体自身能力有限，体现在为智能体配备的工具不齐全，限制了智能体的能力边界。ChatDev 仅仅为智能体配备了 Python 代码执行工具，而没有配备其他工具，比如何网络搜索工具，因此双智能体间的对话难免受到过时的内部知识的局限。又比如 MetaGPT 中质量保证智能体负责软件测试，但是没有为之配备对桌面软件的图像理解和用户界面操控工具，因此其无法有效承担软件 UI 交互操作正确性的验证工作。

多智能体协作缺陷主要体现全流程对话协作固有冗余、智能体角色的冗余、标准化操作流程中的对话缺失、软件开发过程性反馈缺乏：

ChatDev 在软件开发全流程进行对话协作，解决方案需要从对话消息中抽取，忽略了智能体自身的工具调用能力。

MetaGPT 中的架构师和项目经理产出的文档中都包含了当前软件依赖包和项目结构的重复内容。

MetaGPT 中每个智能体仅负责依照订阅的消息执行预定义的动作，缺乏明确的双智能体对话协作的灵活性。

软件开发过程性反馈缺乏，比如 MetaGPT 中的需求分析和系统设计阶段缺乏成果反馈机制，未能充分利用其他智能体成员对其工作成果的反馈意见以改善这两个阶段的工作。MetaGPT 中的代码工程师仅负责编写、运行、Debug 功能性代码而没有进行单元测试，因而在代码编写阶段缺乏重要的测试反馈。

人类用户无法参与或干预除了多智能体软件开发全流程中除了需求分析阶段外的任何环节。

除了上述的缺点和不足之外，涉及到大量智能体的多智能体协作软件开发同样面临多智能体带来的高昂的通信成本。比如 MACNet 中 50 个节点的网状拓扑包含了超过 1000 多条边，因此将产生高达数千次的对话协作。在复杂的软件开发下场景下，负责汇总融合的智能体的上下文将很可能无法容纳几十种完整的软件方案，何况除劣集优。

除此之外，目前的多智能体软件开发框架还在安全性上存在显著不足，主要体现在软件开发过程中的代码运行在用户的本地机器上，如果工程师智能体或者质量保证智能体产出的代码中包含恶意代码，将可能会对用户机器的文件系统和其他配置产生严重损害。

1.3 本文结构

本文主要分为四个部分：

第一部分文献综述，指明多智能体软件开发协作方法的研究意义，同时对国内外研究现状进行简要分析，指出当前研究存在的不足。

第二部分相关技术介绍重点介绍了基于大语言模型的智能体技术范式和多智能体系统结构组成，同时介绍了流行的智能体实现框架 LangChain 及其核心构件。

第三部分软件开发多智能体协作方法详细讲解了本文提出的 AgentTeam 的协作方法、软件开发流程，以及其他智能体必要组件。

第四部分实验设计与结果分析介绍了基线与实验设置，详细分析多了对比实验结构，同时结合具体软件开发任务讲解了 AgentTeam 的协作过程，证明协作机制的有效性。

1.4 本文主要工作

目前的多智能体软件开发协作方法，或者说多智能体软件开发框架，如 1.2 节所述，存在智能体角色冗余、某些开发阶段中缺乏反馈机制、软件开发环境不安全等缺陷。为此，本文提出一种智能体角色紧凑的、软件功能可用性有效性反馈密集的、智能体运行环境安全隔离的多智能体软件开发协作框架 AgentTeam，致力于实现自动化软件开发过

程的安全性、最终软件成果的可用性得到有效保证。

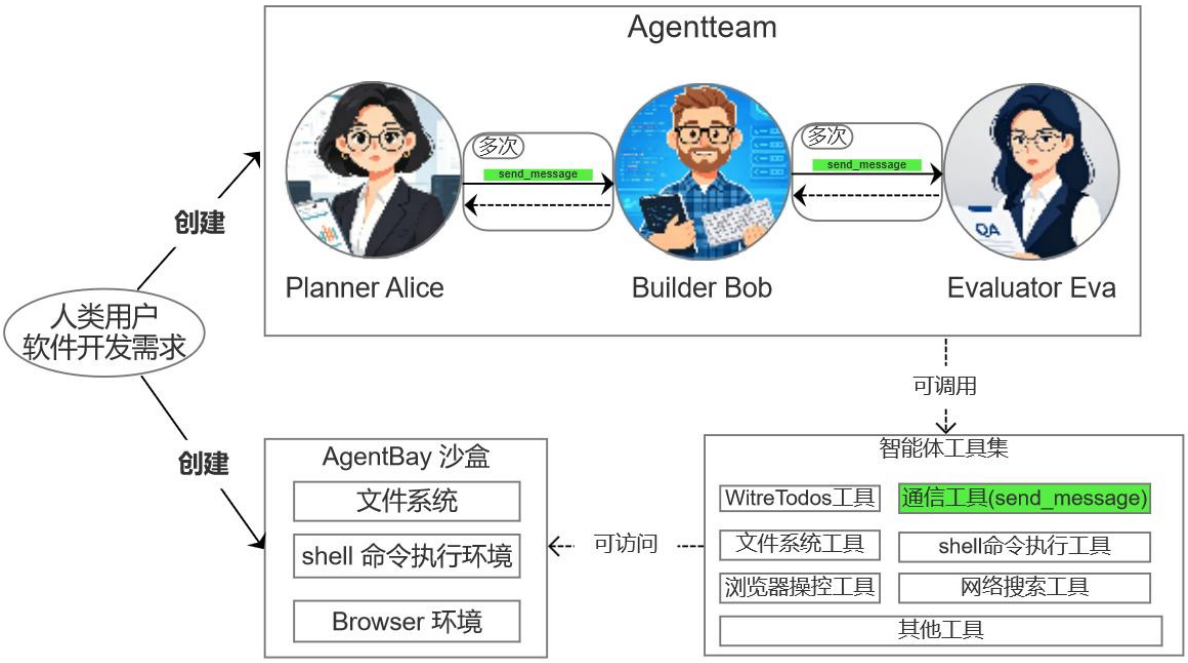


图 1.1　多智能体软件开发协作框架 AgentTeam

1.5　本章小结

大型语言模型（LLMs）对包括软件工程（SE）在内的多个领域产生了重大影响。许多近期出版物探讨了 LLM 应用于各种软件工程任务。尽管如此，关于大型语言模型在软件工程中的应用仍处于早期阶段[20]。ChatDev 虽然可以胜任平均 4.39 个文件左右、144 行左右代码量的简单软件项目的全自动开发，但是还无法承担起现实世界的复杂应用开发任务。MetaGPT 虽然可以在一定程度上处理平均 4.71 个文件、191 行左右代码量的简单软件项目的自动开发，但平均每个项目仍然需要 0.51 次人为代码纠正，还不能完全承担起现实世界的复杂应用开发任务。

总的来说，现有的多智能体开发框架在单智能体能力边界、软件开发流程规范化程度、智能体长短记忆协同、开发任务并行化处理、人在循环支持等方面存在许多不足，距离独立肩负起真实世界软件开发任务而无需任何人类干预还有很长一段距离。

2 相关技术介绍

2.1 大语言模型

语言建模一直是机器学习、自然语言处理的核心任务，目的是对人类自然语言的概率分布进行建模，传统的语言建模技术，比如传统循环神经网络、门控循环单元、长短期记忆神经网络往往面临训练时梯度消失、长期依赖捕捉能力弱，推理时串行化计算效率低等难题。为解决神经翻译任务而提出的 Transformer 架构，创造性地提出了自注意力和多头注意力机制，该机制极大地提高了语言模型捕捉自然语言统计分布规律的能力，同时解决了传统语言模型串行计算效率低的难题，为促进后续大语言模型甚至视觉、语音等其他机器学习领域的发展提供了技术范式。

近些年来，大语言模型技术得到研究者们的广泛关注，取得大量进展，大语言模型的自然语言理解能力和提示响应能力大幅度提升。GPT（Improving Language Understanding by Generative Pre-Training）证明了，通过先在无标注的各式各样的预料库上预训练生成式语言模型，之后在特定自然语言任务（比如文本蕴涵、问题回答、语义相似性评估和文档分类等）上进行区分式微调，可以显著提升语言模型在这些任务上的表现。GPT-2（Language Models are Unsupervised Multitask Learners）证明了大型预训练语言模型能够学习自然语言序列中演示的任务，以便更好地预测它们。GPT-3（Language Models are Few-Shot Learners）证明了将语言模型规模化可以极大地改善其在小样本提示下的表现，证明了上下文学习的有效性。

后续对语言模型的研究发现了名为“涌现能力”的特性，探讨了对语言模型进行额外的规模化是否可能进一步扩展语言模型的能力范围的问题。此外，还有研究者为了提升模型规模以提升模型性能，提出了混合专家架构，以便在有限的计算资源预算下，用更少的训练步数训练一个更大的模型。

2.2 智能体技术

智能体本是强化学习中的技术术语，指的是能够依照一定策略，在某个状态下采取某个动作，收获一定奖励的实体，其智能之处在于能够从轨迹中进行学习，不断优化策略以最大化期望回报。传统的强化学习技术面临奖励模型设计困难、策略泛化迁移能力弱等问题，难以应对真实世界中的复杂问题。

大型语言模型通过预训练，掌握了大量的世界常识，这些世界知识是难以通过传统的强化学习技术得到的。CoT 技术首次证明了链式提示法可以显著激发大语言模型的推理能力[21]，加之后续的思维树[22]、思维图[23]、自一致性[24]等变体技术，进一步表明了

大语言模型作为智能体大脑的可行性。为此，有研究者将大型语言模型作为智能体策略，开展了广泛的基于大语言模型的智能体的研究[25]。

基于大语言模型的智能体各式各样，在近几年取得了大量进展。ReAct 智能体范式证明了通过充分发挥思维与动作的协同效应，可以使大语言模型智能体有效完成真实世界中的复杂任务[26]；Reflexion 提出反思机制，证明了口头反思可以有效提升智能体在多轮尝试某个任务后的任务成功率[27]；Voyager 将大语言模型智能体应用到 Minecraft 中，探索了其在开放世界中执行任务的能力[28]。Toolformer 提出了让大语言模型自己教授自己学会使用各种外部工具的方法[29]，为训练大语言模型的工具调用能力奠定了技术范式。

2.3 多智能体系统

多智能体系统充分借鉴人类社会的分工协作理念，试图通过智能体间的协同推理提升任务解决能力。多智能体系统的构建涉及多智能体运行环境、智能体配置、多智能体通信和智能体能力获取，其整体结构如图 2.1 所示。

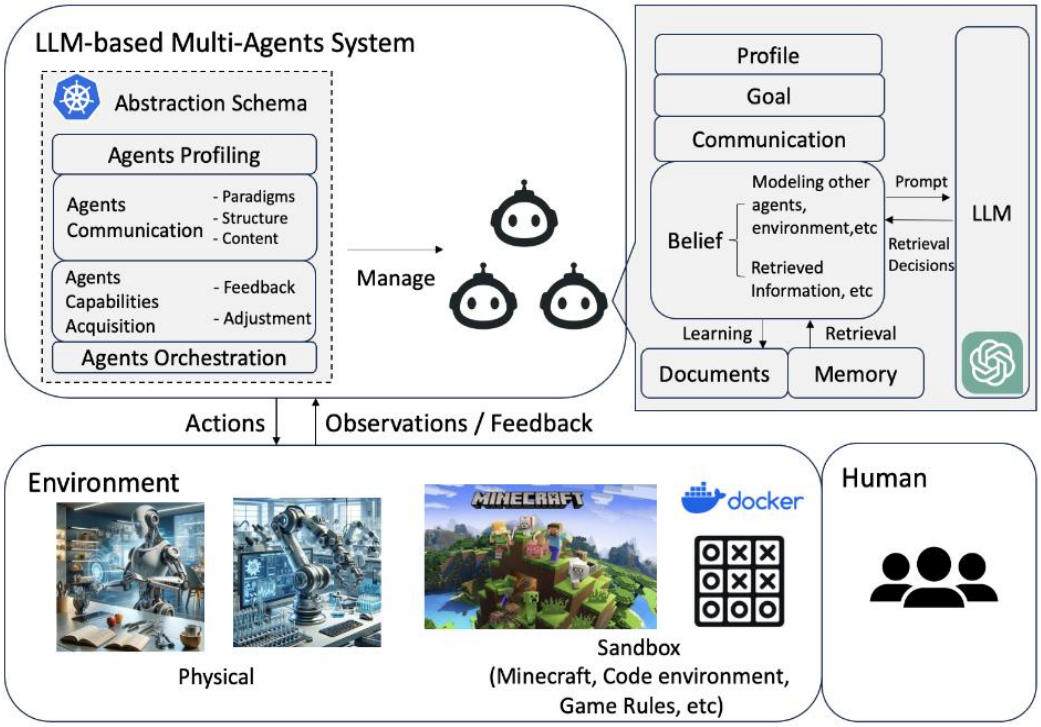


图 2.1 基于大语言模型的多智能体系统整体结构图[30]

多智能体运行时环境定义了多智能体的动作的施加对象以及动作反馈的来源，常见

的运行时环境包含人为构建的虚拟环境（沙盒）、物理环境（真实世界）。ALFWorld 的文本版本就是典型的沙盒环境，它的文本游戏引擎能够允许智能体执行文本表示的动作，同时将文本表示的环境反馈返回给智能体，又或者基于 Minecraft JavaScript API 构建的游戏环境也是典型的沙盒环境，允许将智能体的文本动作转化到相应的 API 调用，进而体现为游戏内角色对环境的改变。物理环境主要用于机器人智能体的相关研究，其中智能体需要和真实世界中的实体交互，同时要遵循真实世界的物理法则和各种其他约束。还有一类特殊的运行时环境就是最简单的消息池环境，其中多个智能体的主要活动就是对话讨论以达成共识，而不关注是否有外部环境的反馈。

智能体配置主要涉及智能体的系统提示词的定义，是将大语言模型从局限于处理对话聊天任务提升到完成真实世界任务的重要环节，通常包含了角色、能力、行为、约束、技能等方面的全面描述，主要的实现方法包含人类手工定义、大语言模型生成、基于真实人类信息生成。

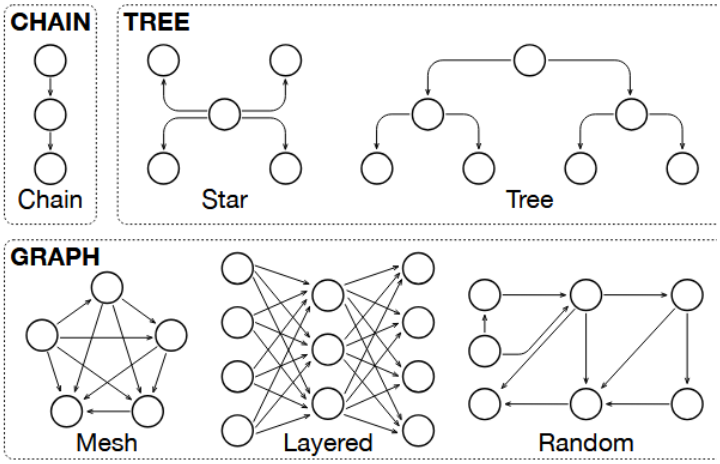


图 2.2　MACNet 提出的 6 种主流的多智能体协作拓扑结构[19]

多智能体通信是支持集体智慧的关键基础设施，主要包含通信范式、通信结构。通信范式主要包含合作、辩论和竞争。

合作通信范式下，多个智能体共同努力实现一个或多个共同的目标，通常交换信息以增强集体解决方案，AgentVerse 中的水平通信结构是这一范式的典型案例。

辩论范式要求智能体对于其他智能体的观点和解决方案，必须提出并捍卫自己的观点和解决方案，并批评其他人，这种范式是达成共识或更精确的解决方案的理想选择。竞争范式下，单个智能体朝着自己的目标努力，而这些目标可能与其他代理人的目标相冲突。

通信结构定义了多智能体系统中智能体之间交换信息的顺序和单个智能体可交流的其他对象，MACNet 将各种智能体协作拓扑统一表示为有向无环图，主要分为中心化的星形结构、去中心化的对等网状结构、串行的链状结构、多层感知机状的分层结构，如图 2.2 所示。

串行链状主要适用于软件开发这种具有明确阶段划分的复杂任务，星形结构主要用于一个中心编排者管理多个下级智能体的场景，网状和分层结构在需要多样化观点的讨论任务上表现较好。

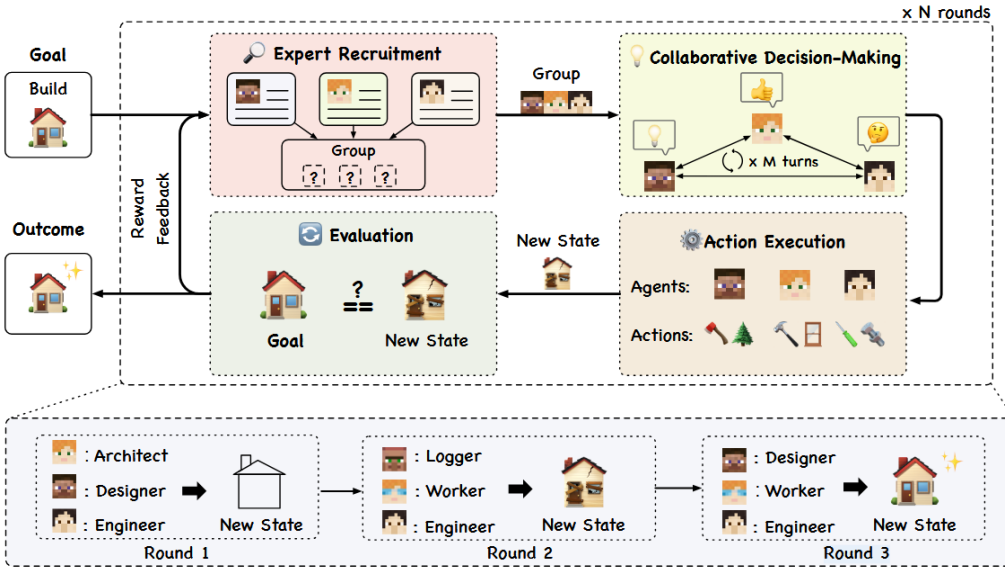


图 2.3　AgentVerse 中每轮任务解决之初存在动态的专家智能体招募阶段[18]

智能体能力获取是多智能体系统中的关键过程，赋予智能体不断学习和动态演化的能力。多智能体系统在持续运行中，每个智能体通常会持续收到各种类型的反馈，比如来自外部环境的反馈，和其他智能体交互得到的反馈，直接来源于人类的指导反馈，每个智能体在理想情况下，应该能够采取合适的适应策略利用这些反馈调整自己，以提高自己处理复杂问题的能力。多智能体系统中常用的适应策略包含记忆、自我演化、动态生成。记忆涉及存储哪些从反馈中提取的信息，比如执行任务的成功轨迹或失败轨迹，并且在面对新任务时自动检索出记忆信息。记忆条目在检索时的评价分数可以同时考虑语义相关性、重要性、时效性等因素，该方法在生成式智能体模拟人类社区时被使用。自我演化涉及更改智能体配置（比如其中的初始目标、规划策略等内容），甚至是基于反馈（这些反馈可以来源于智能体和外部环境、外部工具的交互）微调作为智能体大脑的大语言模型。动态生成方法则是指多智能体系统可以运行过程中即时生成新的智能

体，比如 AgentVerse 在解决一个任务每轮运行之处的专家召集阶段，会根据当前成果状态和进展阶段动态调整本轮的专家智能体团队的成员组成，新建智能体加入到专家团队中，如图 2.3 所示。动态生成方法引入了专门为解决当前需求和挑战而设计的代理，使得多智能体系统能够有效地扩展。

随着多智能体系统中得智能体数量不断扩展，从最简单的双智能体持续对话解决问题，到由 50 个智能体组成的复杂网络结构中的超 1000 次智能体交互，多智能体系统正不断朝着规模化扩展，如何有效编排智能体进行协同推理成为备受关注的问题。

2.4 智能体实现框架 LangChain

LangChain 是一个高度灵活、生态成熟的智能体实现框架，它提供了预构建的智能体架构和模型继承，可以帮助人们将大型语言模型无缝融合到个人应用中[31]。LangChain 针对不同模型提供商有各自独特的 API 的问题，专门提供了标准化的模型接口，同意了模型的响应格式，标准化了用户与模型的交互方式，使用户能够无缝切换大语言模型提供商。除此之外，LangChain 遵照强大的 ReAct 智能体范式，如图 2.4 所示，提供了预构建的 ReAct 智能体，使得用户能够在 10 行左右构建一个简单的智能体。

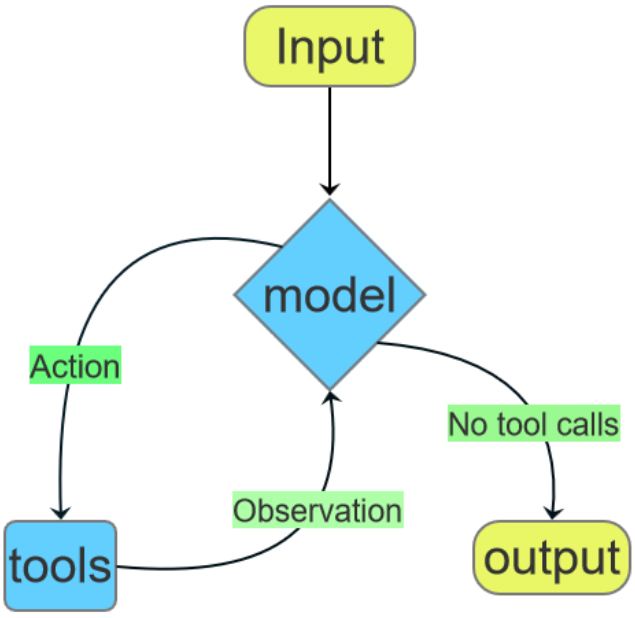


图 2.4 ReAct 智能体结构

原始的 ReAct 智能体每次启动任务时，首先接收原始任务要求，之后做出动作，动作执行并对环境造成影响，知乎智能体获得从环境中得到的观察，作为它思考下一步行动的依据，如此循环，直到任务被解决或者推理-行动循环次数超出预设阈值。

在 LangChain ReAct 智能体实现下，模型一轮 ReAct 循环涉及多轮的模型响应和工具调用，ReAct 循环的结束条件就是模型判断可以给予用户最终响应并且该响应中不包含任何工具调用。

LangChain Agents 是 ReAct 智能体的主要实现，但是却不仅仅是简单的原始 ReAct 智能体。由于 LangChain ReAct 智能体底层基于 LangGraph，因此具备了耐久执行、人在循环等强大特性。更重要的是，由于 LangGraph 以自定义的图形拓扑组织大型语言模型应用的执行过程，因此可以通过实现智能体中间件，增删节点或者编辑节点的内部运行，以此增强传统的 ReAct 智能体。如图 2.5 所示，增加的节点可以是 before_agent，这个节点在每轮 ReAct 循环的最开始执行一次，或者是 after_agent，在每轮 ReAct 循环前结束前执行一次。before_model 和 after_model 分别在 ReAct 循环进行中的每次模型调用前以及模型调用后执行一次。对于 warp_model_call 和 wrap_tool_call 它们则是每次 model 节点和 tool 节点执行时执行一次。

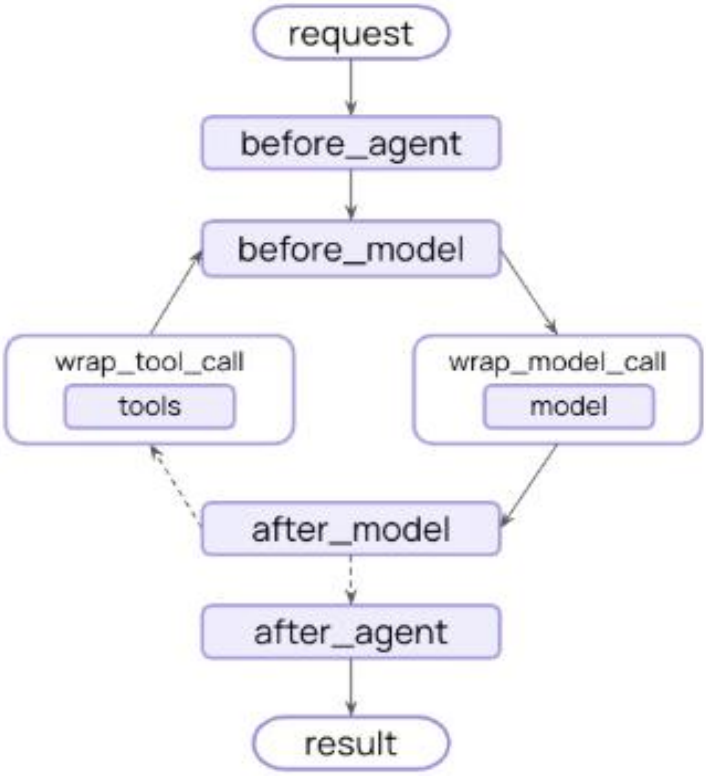


图 2.5 LangChain AgentMiddleware 原理图[31]

LangChain AgentMiddleware 的核心功能在于在合适时机为大语言模型提供合适的信息，这些信息应该有助于帮助智能体解决当前任务。比如 SkillsMiddleware 可以赋予

智能体技能系统，允许智能体感知它当前可加载并使用的工具有哪些，这就是通过增加一个 before_agent 节点函数实现的，在该节点函数中，需要解析智能体创建时添加的所有技能目录来源，对每个技能目录进行解析，提取技能的元信息，即名称、描述和文件路径，并将其注入到大语言模型的系统消息中。又或者 LLMToolSelectorMiddleware，用于智能体面对大量工具并筛选的场景，其主要通过 wrap_model_call 和 wrap_tool_call 发挥作用，在模型调用之前，先调用一次 LLM，询问他目前那些工具和当前查询最相关，一次过滤无关工具以减少词元消耗，可以提升模型注意力和响应的准确性。

2.5 本章小结

大模型技术的正在不断迭代升级，不仅仅是文本理解和推理能力，多模态理解和推理能力也在不断提升，大模型将越来越逼近人类的感知和推理水平，从而承担起更加复杂和现实的任务。

随着智能体大脑的发展，智能体的能力也将得到大幅提升。尽管模型能力的不断发展增强，智能体系统、多智能体系统中的提示词工程、上下文工程和机制过程（赋予智能体进化能力的系统机制）仍然不可或缺[32]，并且将在未来新的任务场景中和大模型技术一同不断进化发展。

3 软件开发多智能体协作方法

3.1 多智能体团队概述

本文提出的智能体开发团队是一个智能体角色紧凑的、协作紧密的多智能体软件开发团队，本文称之为 AgentTeam，若无特殊说明，后续“智能体开发团队”专指本文提出的 AgentTeam。现将 AgentTeam 的各团队成员简单介绍如下：

Planner Alice：专业的软件需求分析师和开发任务规划师，用户与智能体开发团队的唯一接口，负责接收原始用户需求和指派软件开发任务到团队成员。

Builder Bob：全栈开发工程师，软件的主要实现者，负责软件项目创建和软件功能开发。

Evaluator Eva：软件实现的评估者，作为严格的软件质量保证工程师和技术评审专家，负责软件质量保证工作和软件开发推进状态维护。

3.2 智能体能力获取和增强

3.2.1 多样化工具

大语言模型，作为智能体的大脑，具备人类水平的决策能力。工具，作为智能体的手脚，是扩展智能体能力的重要手段。为使智能体开发团队具备充足的软件开发能力，本文为 Builder 和 Ecaluator 配备了强大的开发工具。

为了充分确保 Builder 能够稳步推进软件功能点实现，除了基本的文件系统工具（比如 ls, read_file, write_file, edit, glob, grep）以外，Bob 能够访问一个 execute 工具，调用该工具允许它在当前沙箱环境中执行 shell 命令，从而完成项目虚拟环境创建、依赖包安装、代码文件运行等常见开发任务。

表 3.1 常用浏览器页面级操作工具

工具	功能	适用场景
sandbox_page_use_navigate	导航到 URL	打开应用页面
sandbox_page_use_observe	观察页面元素	查找可交互的输入框、按钮
sandbox_page_use_extract	提取页面内容	验证结果、检查提示信息
sandbox_page_use_act	执行交互操作	点击、输入等复合操作

需要注意的是，所有工具的执行均和某个运行时沙箱绑定，所以调用时需要由智能体传入 sandbox_id。其中以 sandbox_page_use_开头的工具相较于底层浏览器工具的高级

之处在于其中某些工具接收自然语言指令作为实参，比如，比如 page_use_observe、page_use_extrac 的 instruction 形参和 page_use_act 的 act 形参。

表 3.2　常用底层浏览器工具

工具	功能	适用场景
sandbox_browser_click	点击元素	直接点击指定元素
sandbox_browser_type	输入文本	向输入框填入文字
sandbox_browser_wait_for	等待时间	等待页面刷新/加载
sandbox_browser_evaluate	执行 JS 代码	处理复杂交互、动态值设置
sandbox_browser_press_key	按键操作	刷新页面（F5）等

表 3.3　桌面 UI 控制工具

工具	功能
sandbox_list_root_windows	列出所有窗口
sandbox_get_active_window	获取活跃窗口信息
sandboxbox_maximize_window, sandbox_minimize_window, sandbox_close_window`	窗口操作（最大化/最小化/关闭）
sandbox_click_mouse, sandbox_move_mouse, sandbox_drag_mouse	鼠标操作
sandbox_press_keys, sandbox_input_text	键盘操作

为了确保 Builder 能够通过运行验收测试用例获得有效的反馈，Eva 将强制要求验收标准提议中的每个测试用例必须是可通过运行验证、有明确输入输出的用例，大致分为 UI 交互操作测试用例和接口测试用例。由于大语言模型智能体本身不具备任何视觉能力，面对 UI 为 Web 页面的软件开发任务下的 UI 交互操作测试用例难以避免地无法胜任，因此本文为 Builder Bob、Evaluator Eva 配备了浏览器自动化操作工具，这类工具赋予了它们对浏览器页面元素的视觉理解和细粒度操控能力，从而能够从 UI 交互操作测试用例的执行中获得改进当前代码实现的有效反馈。

具体来说，本文为智能体软件开发团队配备了 Wuying AgentBay MCP 工具，主要包含浏览器页面级操作工具，底层浏览器工具，桌面 UI 交互控制工具。现将其中经常被智能体使用的工具简单列表如表 3.1、3.2、3.3 所示。

3.2.2 技能系统

技能，是一种包含了专业工作流和专业领域知识的一组文本文件，用于为智能体提供可复用的能力，其目录结构如图 3.1 所示。为了确保智能体团队中的智能体在面对复杂任务时，能够充分利用现有的专业知识和标准化工作流程，本文为智能体团队中的 Builder Bob 和 Evaluator Eva 配备了技能系统，具体来说是基于 LangChain Agents 框架的 SkillsMiddleware 中间件自定义 MySkillsMiddleware 实现。

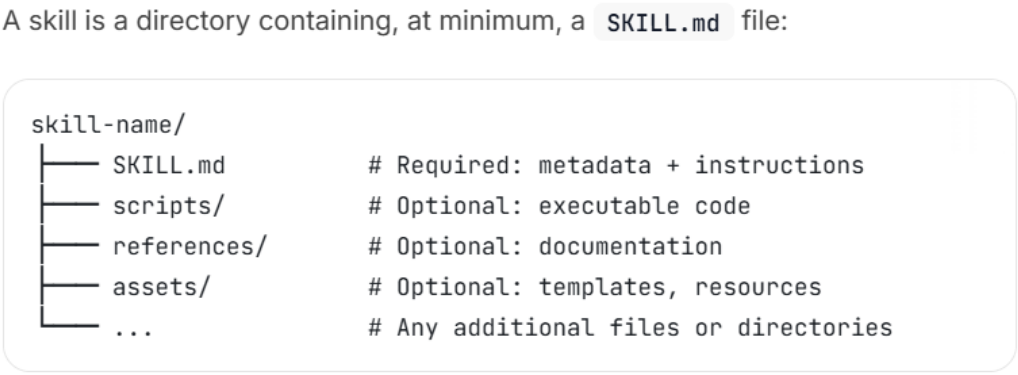


图 3.1 单个技能的目录结构

MySkillsMiddleware 技能中间件通过遵循“渐进式披露”原则来发挥作用，将技能的使用时机交由智能体自主判断。具体来说，技能中间件在初始化时会读取并解析技能文件夹，按照既有的技能标准对其进行判断，比如技能文件夹的命名是否遵循小写并且使用连字符连接不同单词，目录结构是否合理，是否包含必要的 SKILL.md 等等。之后，在每次大语言模型调用前，技能中间件会将解析完成的技能元数据信息（包含技能名称、技能功能自述、技能文件路径）注入到系统提示词中，从而实现了“渐进式披露”，即智能体能够先看到技能名称与简介，仅在它判断执行当前任务需要某个技能时，再去查阅某技能的完整使用说明。

作为智能体开发团队中的核心角色，Builder Bob 必须具备必要的软件开发规范知识和软件项目管理知识，考虑到 Python 语言的简洁易用和强大功能特性，本文特地为 Builder Bob 设计了 python-full-stack-development 技能，明确指导了智能体如何进行 Python 项目虚拟环境创建和使用镜像源进行依赖包安装，此外还对依赖版本锁定、代码质量等最佳实践进行了说明。

为了提高 Builder Bob 和 Evaluator Eva 执行验收测试用例的效率，本文为它们配备了 software-function-verification 技能，重点指导了如何高效利用浏览器操作工具和桌面

UI 控制工具完成测试用例的执行和结果获取。

3.3 多智能体团队协作

3.3.1 开发工作流

多智能体协作涉及多智能体通信，需要明确智能体通信的时机和对象。在多智能体软件开发场景下，需要定义相应的软件开发工作流，相较于 ChatDev 或 MetaGPT 中硬编码每个智能体的通信时机（触发条件）和通信对象，本文提出的智能体开发团队的开发工作流的定义分散在各个团队成员智能体的系统提示词中，工作流的实际执行则通过智能体多次主动调用 send_message 工具实现。

整体上来说，智能体团队的软件开发工作流如图 3.2 所示。

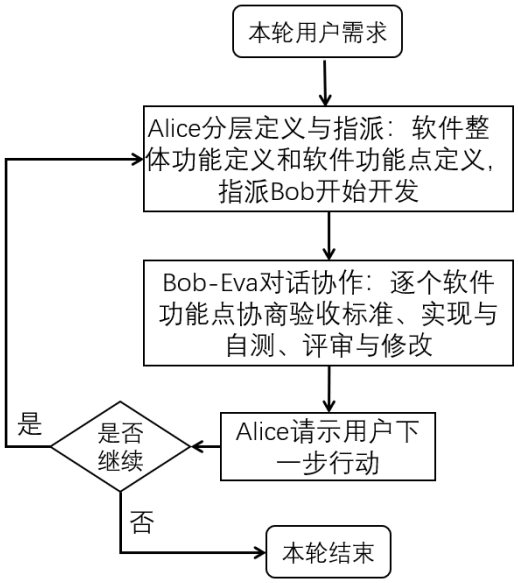


图 3.2 AgentTeam 软件开发工作流示意图

首先，用户向 Planner 提出软件开发请求，Planner 收到后软件需求后，开始进入定义阶段，定义阶段负责完成软件整体功能定义和软件功能点定义，两者都分为“基础版”、“进阶版”、“顶级版”三个层次。分为三个层次、多个软件功能点一方面可以驱使智能体开发团队分步实现，另一方面为用户提供了检验软件成品功能的对照列表，如果未来可能发生人机协作，此举将大大降低用户自身的技术门槛。

Planner 完成定义阶段后，进入任务指派阶段，每次任务指派涉及将一批软件功能点指派给 Builder Bob，Bob 收到本次需要实现的软件功能点列表后，开始逐个软件功能点推进软件开发工作。

每个软件功能点的开发需要 Bob 和 Evaluator Eva 两者进行紧密协作。对于每个功能点，分为验收标准协商、代码生成与自测、功能点实现验收三个阶段。

在验收标准协商阶段，Bob 负责提出验收标准提议，提议中主要包含必要实现项和验收测试用例；Eva 收到提议后按照既定标准对提议进行审核并向 Bob 反馈修改意见，Bob 收到反馈后自行修改后再次提议，直到提议评审通过并进入下一阶段。具体协作流程如图 3.3 所示。

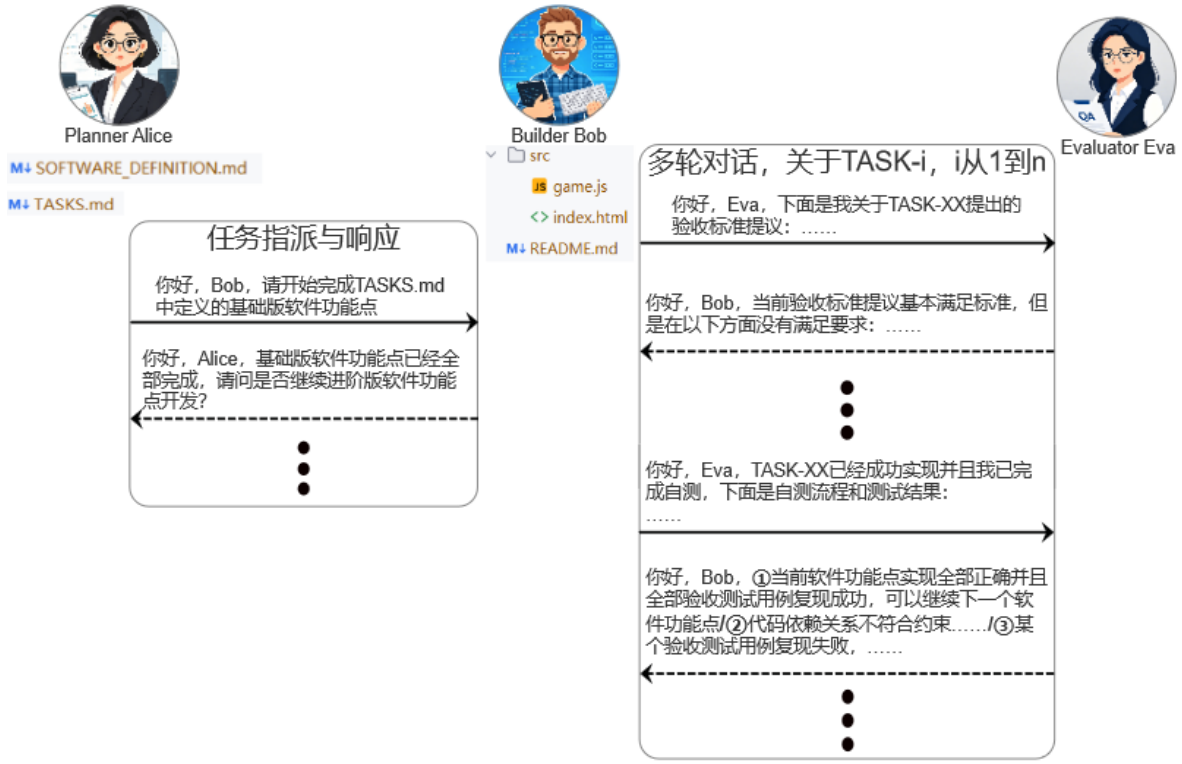


图 3.3　协作流程示意图

在代码生成与自测阶段，Bob 开始项目搭建、依赖包安装、代码文件创建。在代码就绪后，Bob 将逐一执行既定的验收测试用例，并依据测试反馈修改实现代码。

在功能点实现验收阶段，Bob 将当前功能点的实现代码文件和测试用例执行结果告知 Eva 请求评审，Eva 将依照既定标准实行代码静态审查，代码审查通过后接着进入既定验收测试用例复现阶段，验收测试用例全部复现成功后 Eva 将更改当前软件功能点的状态为已完成并通知 Bob 开始推进下一个软件功能点。

3.3.2　任务分解

任务分解是人类面对复杂多步问题时的惯用策略，在软件开发场景下，一个软件系

统可以被分解为多个模块，不同模块交由不同的开发团队或个体人员负责。任务分解也是智能体任务执行能力提高的一大重要因素，因为任务分解本身的输出就能很好地规范基于大语言模型智能体在特定上下文下的输出，进而改善其推理结果的正确性、合理性。

本文提出的智能体软件开发团队的任务分解体现在两个维度，第一是面对来自用户的原始软件需求的开发任务分解，第二是每个智能体任务执行过程中的复杂任务的分解。

为了确保智能体开发团队的软件开发过程稳定推进，而不至于过度开发，本文在 Planner Alice 的 Inception Prompt（或者说 System Prompt）中明确说明了它的任务分解职责。具体来说，要求 Alice 在接收到用户原始软件需求后，必须先进行软件整体功能定义和软件功能点划分，并且两者的划分层次都必须包含“基础版”、“进阶版”、“顶级版”。三个划分层次的理念来自于现实世界中各种软件产品或服务乃至实物产品或服务的分级实践，比如 Starter、Plus、Max 分级。

一般来说，“基础版”的软件成品应该包含本软件的必备核心功能，“进阶版”和“顶级版”则应该在“基础版”软件功能点的基础上聚焦提升用户体验，增加更多富有吸引力的功能。值得注意的是，在本文看来，软件整体功能定义和软件功能点定义对照的是现实世界软件开发团队中产品经理的角色，正如 Planner Alice 的角色定义那样，因此软件整体功能定义的每个层次、每项软件功能点都应该仅仅关注用户可直观感知的功能，而不应该包含任何技术细节。但是，如果用户原始需求中明确定义了技术约束，比如该软件必须使用某种编程语言、某个技术框架、某个第三方软件包，那么 Planner Alice 有责任在向 Builder Bob 转述软件功能点实现任务时明确指出用户表达的技术约束。

软件功能实现是一项长周期的复杂任务，需要智能体开发团队的每个成员，尤其是 Builder Bob，对当前的任务进度保持紧密的跟踪。为此，本文为智能体开发团队的每位成员配备了 write_todos 工具，要求每位智能体合理使用 write_todos 工具来协助管理和规划复杂目标。该工具对规划复杂目标、将大型复杂目标拆解为更小步骤非常有用。同时通过系统提示词注入，要求智能体在完成某一步骤后，立即将对应待办项标记为已完成而不要在完成多个步骤后才批量标记。

3.4 沙盒运行环境

为了确保智能体开发团队的整个开发过程安全可靠，不会危害用户本地机器的既有文件系统和配置，整个智能体开发团队将置于一个隔离的计算机沙箱环境中。对于每个新启动的软件开发任务，都将为之创建一个单独沙箱环境，不同任务在沙箱中具备自身的项目空间，从而有效避免了历史软件开发任务的成果物被后续其他任务中的操作污染

的风险。每个沙盒环境基于一个 MCP 会话，其生命周期如图 3.4 所示。

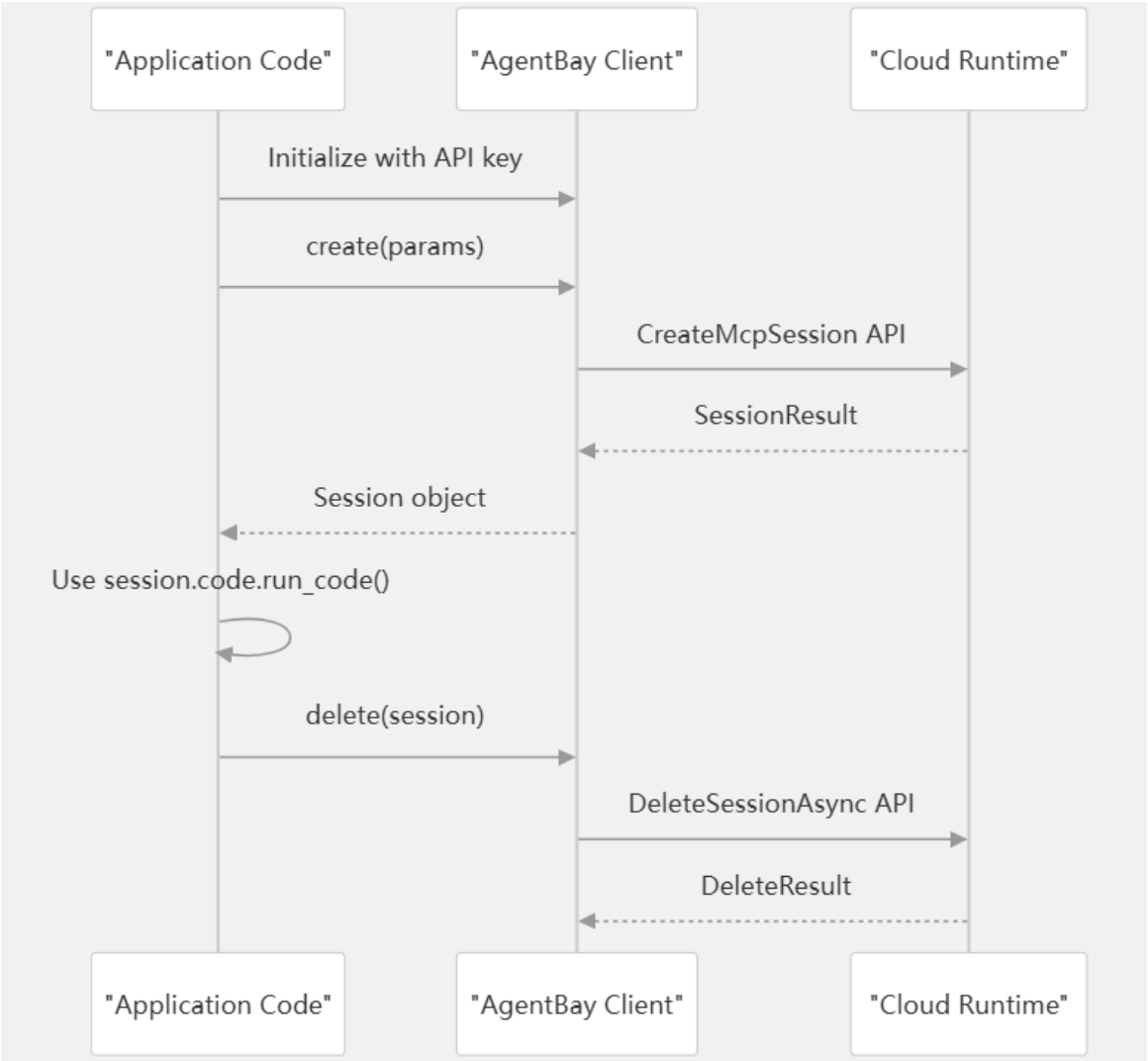


图 3.4 AgentBay 沙盒会话生命周期[33]

本文的沙盒系统基于阿里云 Wuying AgentBay Python SDK 开发套件和 LangChain DeepAgents BaseSandbox 接口实现，同时为了给予智能体开发团队一个合适的开发环境，本文特地基于 Ubuntu 2024 镜像创建了一个自定义镜像。考虑到常见软件开发场景下的运行时环境，比如 Node.js、Python，本文在自定义镜像中预装了 Python3.10-env，为智能体 Builder Bob 在开发 Python 项目时创建项目虚拟环境提供便利。

值得注意的是，在每次软件开发任务启动时创建的沙箱环境并不是持久存在的，本文为之预先设定了 2 个小时的最大运行时间，同时设定该沙箱在 30 分钟内无有效活动后自动释放，一方面对智能体软件开发团队单次任务的最大开发时间做了限制，另一方

面也考虑了程序崩溃后云端沙盒资源应该及时自动释放的问题。

3.5 实现细节

3.5.1 通信工具

正如本文相关技术介绍-多智能体系统一节中所述，多智能体通信是支持集体智慧的关键基础设施，是充分发挥智能体团队集体智慧的关键。相较于基于共享消息池的消息发布订阅机制等异步通信方法，本文考虑到采用智能体团队进行软件开发，相较于人类开发团队，具备超高效率，因此本文采取简单的“智能体即工具”的理念，将 Builder Bob 和 Evaluator 各自封装在一个工具中。

具体来说，本文为 Planner Alice 和 Builder Bob 配备了一个名为 send_message 的工具，该工具接收 expert_agent 和 private_message 两个参数，每当主调智能体调用该工具，工具执行时将会封装 private_message 的具体内容为一条 HumanMessage 并将之作为消息接收者的本次输入，之后消息接收智能体将开启本次的 ReAct 循环。因此，用户发送消息可触发 Planner Alice 的 ReAct 循环，Alice 可以通过调用 send_message 触发 Builder Bob 的 ReAct 循环，Bob 在其自身循环中也可以调用 send_message 触发 Evaluator Eva 的 ReAct 循环。如此实现了多智能体软件开发团队的紧密协作。

在该协作机制中，主调智能体可以通过被调智能体的运行反馈获取软件功能点可用性的有效反馈，从而可以及时修正当前的软件功能点实现的缺陷，确保后续软件开发的起点的正确性。

为确保智能体协作的稳定性，send_message 工具实现了严格的并发控制机制。由于同一专家智能体在同一时刻只能处理一个任务请求，系统采用了双重锁机制来防止并发冲突：

线程锁机制：对于同步版本的 send_message 函数，系统为每个专家智能体维护一个独立的 threading.Lock 对象。当主调智能体尝试调用 send_message 时，首先尝试非阻塞方式获取对应专家的锁。如果锁已被占用，说明该专家正在执行其他任务，系统将立即返回错误信息而非阻塞等待，错误消息明确告知调用者"专家智能体'XXX'正在执行另一个任务，不支持对同一专家智能体并行调用 send_message"。这种快速失败（fail-fast）策略避免了资源竞争和死锁风险。

异步锁机制：对于异步版本的 asend_message 函数，系统为每个专家智能体维护一个独立的 asyncio.Lock 对象。异步版本同样采用非阻塞尝试获取锁的策略，如果锁不可用，则记录警告日志并选择阻塞等待或快速失败。异步锁的使用确保了在异步事件循环中不会出现竞态条件，保证了专家智能体状态的一致性。

3.5.2 消息传递的具体流程和数据格式

end_message 工具的消息传递流程遵循严格的数据封装和解封协议，确保信息在智能体之间准确传递。

(1) 消息封装阶段

主调智能体调用 send_message 工具，传入 expert_name 和 private_message 两个参数。之后_validate_and_prepare_call 函数验证专家名称的合法性，并从 runtime.state 中读取对应的 expert_thread_id。然后，系统构建 HumanMessage 对象，消息内容经过特殊格式化，具体格式为“1. Here comes a message from {sender_name}:{private_message} 2. To respond to {sender_name}, just give your reply to me”。这种格式化明确了消息来源和响应目标，帮助专家智能体理解上下文。最后，专家输入状态 expert_input 被构造为{"messages": [HumanMessage(...)]}格式。

(2) 专家执行阶段

系统构建 expert_config 配置对象，注入 thread_id 和 checkpoint_ns，调用 expert.invoke(expert_input, config=expert_config)启动专家的 ReAct 循环。专家智能体根据自身的 system_prompt 和可用工具执行任务，可能涉及多次 LLM 调用和工具调用，专家的执行过程被其专属的 checkpointer 自动持久化。

(3) 结果提取阶段

专家执行完成后，_return_command_with_state_update 函数从 expert_result 中提取最后一条消息。系统验证 expert_result 包含"messages"键，否则抛出 ValueError 异常。提取 last_message 的内容（支持 content 或 text 属性），并添加前缀"Response from {expert_name}:\n"。构造 ToolMessage 对象，tool_call_id 设置为当前 send_message 调用的 ID，确保结果能正确关联到原始工具调用。

(4) 结果返回阶段

返回 Command 对象，其 update 字段包含{"messages": [ToolMessage(...)]}，之后 LangGraph 框架将 ToolMessage 注入到主调智能体的状态图中，完成一次完整的消息传递循环。

3.5.3 专家智能体的独立状态持久化

send_message 工具的另一个关键设计是实现了专家智能体的独立状态持久化机制。在传统的主从式智能体架构中，所有子智能体的状态通常依附于主智能体的状态图中，这导致了状态管理的复杂性和耦合度高的问题。本文提出的解决方案通过以下技术手段实现了专家状态的完全解耦：

(1) Thread ID 映射管理

CommunicateMiddleware 中间件在主调智能的状态图中维护一个 expert_thread_ids 字典，该字典以专家名称为键、以 UUID 生成的 thread_id 为值。这个映射关系在中间件的 before_agent 钩子函数中初始化，确保每次智能体会话启动时都能正确建立专家与 thread_id 的关联。值得注意的是，expert_thread_ids 被标记为 PrivateStateAttr 类型，表明这是内部状态，不会暴露给用户输入或最终输出。

(2) Checkpoint 命名空间隔离

当 send_message 工具调用专家智能体时，会构建一个专用的配置对象 expert_config，其中包含两个关键字段：

thread_id：覆盖为专家专属的 thread_id，确保专家的状态读写操作指向正确的检查点记录。

checkpoint_ns：设置为"expert:{expert_name}"格式，为每个专家创建独立的命名空间。

这种设计使得不同专家的检查点数据在存储层面完全隔离，即使使用相同的底层存储后端，各专家的状态也不会相互干扰。例如，Planner Alice 的 checkpoint 存储在默认命名空间下，而 Builder Bob 的检查点存储在"expert:bob"命名空间下，Evaluator Eva 的检查点存储在"expert:eva"命名空间下。

(3) 懒加载策略

专家智能体的 thread_id 采用懒加载策略生成。在 CommunicateMiddleware 的 before_agent 钩子中，系统检测到 expert_thread_ids 不存在或为空时，会为所有配置的专家一次性生成 UUID 格式的 thread_id。这种策略避免了在编译时为尚未使用的专家预分配资源，同时也确保了 thread_id 在会话期间的稳定性。

(4) 状态恢复机制

当专家智能体被再次调用时，系统会从主调智能体的 expert_thread_ids 字典中读取已有的 thread_id，而不是重新生成。这确保了专家能够从其上一次执行的检查点恢复状态，实现了跨多次调用的连续记忆。例如，Builder Bob 在完成第一个功能点后，其工作状态（包括已读取的文件、已执行的命令历史等）会被持久化；当 Alice 指派第二个功能点时，Bob 能够从之前的状态继续执行。

3.5.4 智能体状态管理

在复杂的软件开发任务中，智能体需要维持长期的上下文记忆，包括已执行的命令历史、已读取的文件内容、已完成的待办事项等。为确保智能体能够在长时间运行的任务中保持状态一致性，并在意外中断后能够恢复执行，本文设计了一套完善的状态管理

与持久化机制。该机制基于 LangGraph 框架的 checkpoint 系统，结合自定义的中间件和存储后端，实现了多层次的状态管理。

LangGraph 框架提供了内置的 checkpoint 机制，用于保存和恢复智能体的执行状态。checkpoint 本质上是一个序列化的状态快照，包含了智能体在某一时刻的完整上下文，包括消息历史、变量值、工具调用记录等。

本文实现的 checkpoint 系统具有以下特点：

(1) 双后端支持

系统支持两种 checkpoint 后端，可根据部署环境和性能需求灵活选择。AsyncPostgresSaver，基于 PostgreSQL 数据库的异步检查点保存器，适用于生产环境。PostgreSQL 提供了事务保证、数据持久化和并发控制，确保 checkpoint 数据的安全性和一致性。在 AgentTeam 初始化时，若 settings.USE_POSTGRES_CHECKPOINTER 为 True，系统会获取异步连接池，显式打开连接池以避免弃用警告，然后创建 AsyncPostgresSaver 实例并调用 setup()方法初始化数据库表结构。InMemorySaver，基于内存的检查点保存器，适用于开发和测试环境。内存保存器具有极低的延迟，但数据在进程重启后会丢失。当 USE_POSTGRES_CHECKPOINTER 为 False 时，系统使用 InMemorySaver 作为后备方案。

(2) 命名空间隔离

为避免不同智能体的 checkpoint 数据相互干扰，系统为每个智能体分配独立的 checkpoint 命名空间（checkpoint_ns）。主调智能体（如 Planner Alice）使用默认命名空间，而专家智能体（如 Builder Bob、Evaluator Eva）使用"expert:{expert_name}"格式的命名空间。这种隔离确保了：不同智能体的状态可以独立保存和恢复，互不影响；同一智能体在不同线程（thread_id）下的执行记录可以区分；便于调试和审计，可以单独查看某个智能体的执行历史。

(3) Thread ID 管理

每个智能体会话都由一个唯一的 thread_id 标识，该 ID 在整个会话期间保持不变。thread_id 的作用类似于文件系统的路径，checkpoint 系统通过(thread_id, checkpoint_ns)的组合键定位特定的状态快照。在 AgentTeam 中，thread_id 的生成和管理遵循以下规则：对于绿地开发（enable_brown=False），系统生成新的 UUID 作为 thread_id；对于棕地开发（enable_brown=True），系统从任务记录文件中读取最近的 thread_id，实现状态恢复；专家智能体的 thread_id 由 CommunicateMiddleware 在 before_agent 钩子中统一生成和管理。

(4) 专家智能体（Builder Bob、Evaluator Eva）的状态通过独立的 thread_id 和

checkpoint_ns 进行管理

这种设计的优势体现在以下几个方面：

① 状态解耦

主调智能体（Planner Alice）不需要维护专家的完整状态，只需保存 expert_thread_ids 映射。这降低了主调智能体的状态复杂度，提高了系统的可扩展性。若未来需要增加更多专家（如 Architect、Tester 等），只需在 CommunicateMiddleware 中注册新专家，无需修改主调智能体的状态结构。

② 并行执行支持

虽然当前实现禁止对同一专家并行调用，但独立的 checkpoint 命名空间为未来的并行执行预留了可能性。理论上，可以为同一专家的不同任务分配不同的 thread_id，实现真正的并行处理。

③ 调试与审计

独立的 checkpoint 命名空间使得调试更加便捷。开发者可以单独查询某个专家的执行历史，而不受主调智能体或其他专家的干扰。例如，通过查询"expert:bob"命名空间下的所有 checkpoint，可以完整还原 Builder Bob 在某个任务中的所有操作。

④ 资源优化

专家状态的独立管理使得系统可以更精细地控制资源使用。例如，可以设置不同的 checkpoint 保留策略：主调智能体的 checkpoint 长期保留，而专家智能体的 checkpoint 在任务完成后可选择性清理，以节省存储空间。

3.5.5 棕地开发支持

在实际软件开发场景中，用户可能需要中断当前的开发任务，稍后从中断点继续执行。为支持这种场景，本文实现了棕地开发（Brownfield Development）模式，允许智能体从之前的 checkpoint 恢复状态。

(1) 软件开发任务记录文件

系统为每个软件开发任务维护一个 YAML 格式的任务记录文件，位于 registered_tasks/{task_name}.yaml。该文件记录了任务的执行历史，包括：thread_ids：历次执行使用的 thread_id 列表；task_context_names：对应的沙盒上下文名称列表；task_context_box_ids：沙盒上下文的唯一标识符列表；task：任务的元数据（来自 datasets 定义。

(2) 状态恢复流程

当用户启用棕地开发模式（enable_brown=True）时，系统执行以下步骤：检查任务记录文件是否存在。若存在，读取最新的 thread_id 和 task_context_box_name。使用读取

的 thread_id 初始化 AgentTeam，确保 checkpoint 系统能够从之前的状态恢复。使用 task_context_box_name 连接到对应的沙盒上下文，恢复文件系统和运行时环境。智能体从上次中断的 checkpoint 继续执行，消息历史、变量状态等均保持不变

(3) 绿地开发模式

若用户强制启用绿地开发（enable_brown=False），系统会生成新的 UUID 作为 thread_id；创建新的沙盒上下文（task_context_box_name = "{task_name}_{timestamp}"）；清空或覆盖任务记录文件，开始全新的开发会话。

这种双模式设计为用户提供了灵活性：既可以从断点续传，也可以重新开始。

3.6 本章小结

本章主要对智能体开发团队进行了介绍，包括团队人员组成，软件开发工作流，工具和环境，协作机制实现，技能系统和任务分解。

对于每个智能体，同样介绍了其所配备的工具情况，包括共有的一些工具以及角色特定的工具。为了给予软件开发团队一个适宜的运行环境，专门为其配备了沙盒环境，在该环境中，智能体可以直接向文件系统中写入代码文件，从而避免了从对话响应中抽取代码片段的冗余环节。为了实现不同智能体间的高效协作，专门将协作模式定义为任务指派和响应接收，为此将被指派的智能体的运行封装到一个工具之中，提供了简单易用的多智能体协作机制。

同时为了充分利用现有的人类专家知识，特地为智能体配备了技能系统，该系统负责向智能体提供一个技能库，允许智能体按需读取技能并且允许智能体自主修改完善技能文件的具体内容。

此外，还详细介绍了基于 LangGraph 的检查器的智能体状态持久化机制，以及本文为了支持棕地开发所作的工作。

4 实验设计与结果分析

4.1 基线

本文选取多智能体软件开发领域两个经典框架作为基线，分别是 ChatDev 和 MetaGPT。ChatDev 充分借鉴双智能体对话的人机协作范式和瀑布模型，将软件开发组织为连续多个阶段中两个专家智能体间的对话协作，并且在系统测试阶段引入了代码执行反馈。

MetaGPT 将软件开发标准化操作流程融入多智能体协作框架之中，比如说角色扮演激发专业知识、结构化通信、简化的工作流，创建多个 GPT 智能体组成智能体开发团队。其采取消息池订阅与发布机制实现智能体通信，其中工程师智能体能够获取代码执行反馈，同时具备结合历史消息迭代修改代码的机制。

4.2 数据集与评估指标

本文选用 SoftwareDev 数据集子集，挑选其前 7 个软件开发任务作为实验数据集。这七个任务包含 2 个 streamlit 应用开发（crud manage、excel data process）和 4 个难度递增的游戏开发（2048、snake game、brick breaker、flappy bird、tank battle），每个任务都具备强逻辑和复杂流程，可以充分考验智能体开发团队的软件开发能力。

多智能体软件开发的传统评估标准主要分为客观统计数据和人类主观评估。传统的客观统计标准包括软件可执行性、软件生成时间、代码文件总数、软件代码总行数、软件生成耗费 Token 数、生产效率（每行代码耗费 Token 数，该值越低表明生产效率越高）、软件代码（排除注释内容）与用户自然语言需求的语义一致性分数，人类主观评估包括软件代码质量评分、人类参与代码修改次数。

值得注意的是，软件可执行并不代表智能体开发团队生成的软件产品的各项核心功能全部正常并且满足原始用户需求，因此还可以通过人类手工验证软件产品的各项核心功能是否正确实现并且给出可用性评价，可用性评价分为可用 Success 与不可用 Failure。

本文选取软件生成时间、软件代码文件总数、软件代码总行数、软件生成耗费词元数和软件成品的功能可用性验证结果作为主要指标。

4.3 对比实验

本文选取 MiniMax-M2.7 作为主干模型，温度设置为 1.0，选取该模型的原因在于 M2.7 在真实的软件工程中表现优异，包括端到端的完整项目交付，分析日志排查 Bug、代码安全，机器学习等[33]。

本文将 ChatDev、MetaGPT 和 AgentTeam 在 SoftwareDev-7 上进行对比实验，收集统计指标并人工验证各自软件产品的功能可用性，具体实验结果如表 4.1、表 4.2 和表 4.3 所示。

表 4.1 ChatDev 在 Software-7 上的实验结果

任务名	耗时	词元消耗	代码文件数	软件代码总行数	软件成品功能人工验证结果	软件成果可用性评价
Excel data process	5 分 20 秒	15，189	1	37	核心功能全部实现	可用
CRUD manage	13 分 15 秒	45，556	2	271	所有功能都正确实现	可用
2048 game	18 分 50 秒	49，507	4	368	浏览器中打开 <http://127.0.0.1:5000> 后，显示 "TemplateNotFound jinja2.exceptions.TemplateNotFound: index.html"	不可用
Snake game	6 分 35 秒	49，932	5	385	启动游戏后仅能完成单局游戏，并且蛇体移动速度过快，用户体验差	不可用
Brick breaker game	18 分 15 秒	44，930	6	272	困死在游戏开始菜单，无法进入游戏	不可用
Flappy bird game	15 分 0 秒	32，140	4	320	浏览器打开 <http://localhost:8000> 后一片灰暗，所有文件使用了 Python 风格的三引号注释 '"..."'，这在 HTML 和 JavaScript 中是无效的语法	不可用
Tank battle game	9 分 35 秒	109，884	7	862	绝大部分功能成功实现，尽管不支持 WASD 控制移动，但是总体上可以认为成功。	勉强可用
指标平均值	12 分 24 秒	49，591	4.1	359	无	**3/7 可用**

需要说明的是，MetaGPT 基线的词元消耗统计方法是采用估计方法，即字符数除以 3.5，而本文提出的智能体团队和 ChatDev 计量的是真正的 Token 消耗。

此外，关于 MetaGPT 在打砖块任务上的词元消耗信息缺失的问题有必要说明。由

于 MetaGPT 自身的设计缺陷导致其在 Brick Breaker game 上的词元消耗出现统计错误。具体来说，MetaGPT 的统计机制是"尽力估算"而非完整统计——依赖特定代码路径的日志输出。而在打砖块这一任务上，WriteCode.run() 路径的 prompt 没有通过_aask_v1，导致该路径的 prompt tokens 完全丢失。

表 4.2 MetaGPT 在 SoftwareDev-7 上的实验结果

任务名	耗时	词元消耗	代码文件数	软件代码总行数	软件成品功能人工验证结果	软件成果可用性评价
Excel data process	2 分 40 秒	6，466	1	52	全部功能都成功实现	可用
CRUD manage	5 分 40 秒	11，481	1	272	总体来说，全部正确实现	可用
2048 game	35 分 16 秒	29，746	9	1173	所有核心功能正确实现	可用
Snake game	15 分 30 秒	13，973	2	386	开始菜单卡死，无法选择难度等级，进而无法开始游戏	不可用
Brick breaker game	8 分 5 秒	未知	6	919	仅正确实现了从开始菜单进入游戏，但是无法正常开始游戏	不可用
Flappy bird game	11 分 5 秒	12，885	6	409	所有功能全部实现	可用
Tank battle game	15 分 55 秒	26，634	5	843	游戏启动失败，报错 `pygame.error: font not initialized`。	不可用
指标平均值	13 分 27 秒	16，864（已排除打砖块）	4.3	579	无	**4/7 可用**

对于软件成品功能人工验证，本文参照软件成品中的相关文档，比如 MetaGPT 方法下软件产品中的产品需求描述文档，或者是 ChatDev 方法下软件产品中的产品使用手册，以及本文方法下软件产品中的 README 文档或者 TASKS.md 文件，提取出每个软件产品的核心功能自述，并据此展开人工软件功能验证过程。

Software-7 数据集中各项软件开发任务的原始需求表述参见附录 A。两个基线及本文方法产出的软件产品的软件功能人工验证结果详情，本文方法产出的软件成品的用户界面截图详情请参见附录 B。

表 4.3　本文提出的 AgentTeam 在 Software-7 上的实验结果

任务名	耗时	词元消耗	代码文件数	软件代码总行数	软件成品功能人工验证结果	软件成果可用性评价
Excel data process	39 分 37 秒	3，352，866	1	82	需求功能全部实现	可用
CRUD manage	75 分 45 秒	12，034，054	1	211	需求功能全部实现	可用
2048 game	62 分 56 秒	10，699，230	2	496	基础版功能全部实现	可用
Snake game	64 分 0 秒	28，828，095	2	467	核心功能全部正确实现	可用
Brick breaker game	76 分 11 秒	5，231，779	1	559	基础版功能全部实现	可用
Flappy bird game	90 分 51 秒	16，431，748	2	252	地面碰撞检测未实现，但不断闯过管道空隙的核心玩法正常保留	可用
Tank battle game	68 分 20 秒	5，231，779	11	2393	玩家坦克不可发射炮弹，图层顺序异常	不可用
指标平均值	68 分 13 秒	11，687，078	2.8	631	无	**6/7 可用**

如表 4.1、表 4.2、表 4.3 所示，从 ChatDev 和 MetaGPT 两者指标对比结果可以看出

MetaGPT 以 ChatDev 大约 1/3 的词元消耗完成了 7 个软件开发任务，并且可用软件数目比 ChatDev 多了 1 个，表明 MetaGPT 在成本和可行性显著超越了 ChatDev，这与 MetaGPT 论文原文中的对比试验结论一致，同时也揭示出智能体软件开发的某些阶段上采取双智能体对话的冗余性。

虽然在 SoftwareDev-7 这个数据集上，ChatDev 和 MetaGPT 各自的开发流程都已经走完，并且并且在软件开发耗时、词元消耗指标上远远超过本文提出的智能体开发团队，但是它们在最终的软件产品的可用性上很令人失望，分别是 3/7（42.8%）和 4/7（57.1%），与之对比，本文提出的智能体软件开发团队在 SoftwareDev-7 上的可用性指标是 6/7。

对于不可用的坦克大战开发任务，AgentTeam 在 TankBattle 这个软件开发任务上选择的产品模态是桌面 GUI 游戏而不是 Web 页面游戏，因此软件开发团队无法利用现有的桌面控制工具充分理解游戏图层的堆叠顺序，同时也无法利用视觉工具感知到玩家坦克发射的炮弹动画是否成功绘制。为此，笔者将坦克大战的原始软件需求从“创建一款坦克大战游戏”更改为“创建一款 Web 版坦克大战游戏”后重新要求 AgentTeam 进行开发，本次 AgentTeam 充分尊重用户原始需求中的成品形态要求，选择基于 HTMl 进行 UI 展示并且使用 JS 进行逻辑编写。在实验完成后，笔者对实验统计信息和软件成果物功能可用性进行评审，结果如表 4.4 所示。尽管 Web 版的坦克大战由于缺失保护基地的核心玩法仍然不可用，但是 AgentTeam 正确实现了几乎全部的基础版软件功能点，究其软件成果不可用的原因，在于该软件开发任务没有提供任何软件功能描述，导致仅能利用内部知识的 Alice 对于“坦克大战”的软件整体功能定义和软件功能点定义中忽略了保护基地这一核心的软件功能，该缺陷有望通过为 Alice 配备网络搜索工具有效解决。

表 4.4　Web 版坦克大战实验结果

任务名	耗时	词元消耗	代码文件数	软件代码总行数	软件成品功能人工验证结果	软件成果可用性评价
坦克大战	104 分 16 秒	38，734，756	1	1183	基础版功能基本正确实现，但是玩家坦克移动控制异常	不可用

即便表 4.3 中平均每个软件开发任务耗费词元数为 11，687，078，相较于 MetaGPT 和 ChatDev 巨大许多倍，但是实际成本仅仅是每个任务平均耗费 0.013 元。

值得一提的是，Software-7 中的 7 个软件开发任务，无论对于 ChatDev、MetaGPT 还是本文的 Agentteam，它们的实现难度并非完全等同。大体来说，crud manage 和 excel data process 两个软件开发任务，相较于其他 5 个游戏开发任务，要简单许多，因为 SoftwareDev-7 中明确指明了 crud manage 和 excel data process 使用 streamlit 这种前后端一体化的 Web 开发框架并且指明了使用 SQLite 作为前者的数据库，使用 Pandas 作为后者读取 Excel 的方法，相较于其他 5 个软件任务简短的用户需求描述，无疑较大程度上降低了这两个开发任务的开发难度。

由此看来，这两个最简单的软件开发任务 crud manage 和 excel data process 为它们贡献了 28.5%的成功率，若刨除这一点，ChatDev 和 MetaGPT 两者产出的软件成品的可用性比例为 1/5（20%）、2/5（40%），这反映出 ChatDev 和 MetaGPT 在面对较难软件开发任务上的真实水平。

总的来说，AgentTeam 依靠充分的软件功能点定义和串行化软件功能点验收评审，确保了复杂软件开发中软件功能的稳妥推进，确保每一次进步的前提和基础都是经过验证的。

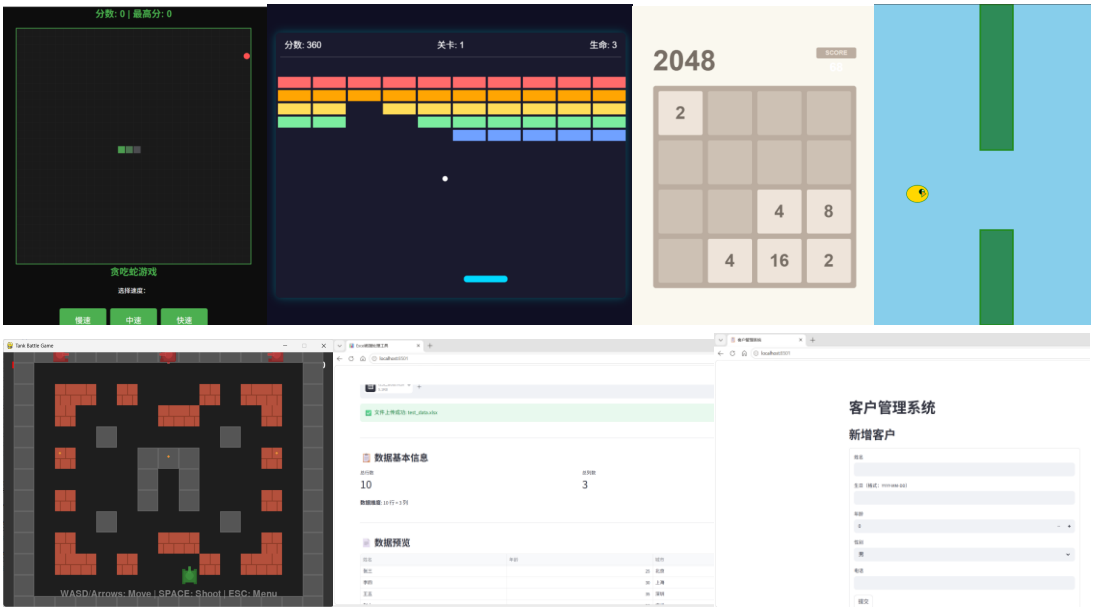


图 4.1　AgentTeam 在 SoftwareDev-7 上的软件成果物 UI 略缩图

AgentTeam 在 SoftwareDev-7 数据集上的软件成果物用户界面略缩图如图 4.1 所示，大部分软件成品的模态为 Web 应用，可见工程师 Bob 能够根据自身可访问的浏览器操作工具明智的的选择了产品模态。此外，绝大多数软件成品不仅功能可用，满足了用户的原始需求，其美术水平也能给予用户较好的使用体验。



图 4.2　Web 版坦克大战软件 UI（左）和相应基础版软件功能点定义（右）

如图 4.2 所示，当在用户需求中明确要求实现 Web 版的坦克大战时，Builder Bob 借助自身的浏览器控制工具很好地完成了基础版软件功能点列表中的所有条目，该软件成品最终不可用的原因在于 Alice 仅依靠内部知识没有准确捕捉到坦克大战游戏中保卫基地这一核心玩法要素。

4.4　角色消融实验

为了了解不同智能体角色对自动化软件开发成果物的影响，笔者额外进行了两项消融实验，逐渐从 AgentTeam 中排除 Planner Alice 和 Evaluator Eva，同样记录了实验统计信息和软件成果物可用性人工验证结果，实验结果如表 4.5 所示。

表 4.5　Agentteam 在 SoftwareDev-7 上的角色消融实验结果

智能体团队组成	任务平均耗时	任务平均词元消耗	软件成果可用性总结
Builder Bob	9 分 27 秒	1，329，157	2/7 可用：2048、Flappy Bird
Builder Bob + Evaluator Eva	45 分 39 秒	9，130，942	3/7 可用：Crud manage、Snake game、Brick breaker

在智能体团队仅有 Builder Bob 单个智能体的情况下，单个软件开发任务耗时相较于完整的 AgentTeam，出现了大幅下降，达到十分钟左右的水平，同时任务平均词元消耗也大幅下降，但是最终软件成果的可用性并没有得到很好的保障。

在智能体团队同时包含 Builder Bob 和 Evaluator Eva 两个智能体的情况下，观察到任务平均耗时以及任务平均词元消耗显著上升，这来源于双智能体对于软件功能点的验收标准协商，以及 Eva 对于软件功能点测试用例的复现过程。

由于单次软件开发任务过程中，每个智能体的消息列表包含几十上百条消息，其中包含种类各异的工具调用消息，人工分析自动化软件开发过程中的失败因素的代价将会

是无法承担的巨大。仅仅从最终软件成果可用性分析，可以看出 Planner Alice 对于初始软件需求的分层定义，以及软件功能点的分层定义对于保障最终软件成果的可用性的作用十分巨大。

4.5 协作过程分析

为什么 AgentTeam 能够很好的处理涉及大量逻辑的游戏开发任务呢？笔者将其归功于 Planner Alice 的细致的软件功能点划分，以及 Builder Bob 和 Evaluator Eva 之间的紧密协作。为了更好地说明这一点，后续将以 crud manage 这一软件开发任务具体说明为什么 AgentTeam 在软件可用性上的表现如此成功。

已知 crud manage 的原始软件需求描述如下：编写一个基于客户业务实体的增删改查（CRUD）管理程序。客户需存储信息：姓名、生日、年龄、性别、电话。数据存储在 client.db 数据库中，需判断客户表是否存在，不存在则先创建。查询和删除操作均通过姓名实现。程序要求简洁、易维护、不过度设计，使用 Streamlit 和 SQLite 实现界面功能，无需引入其他额外包。

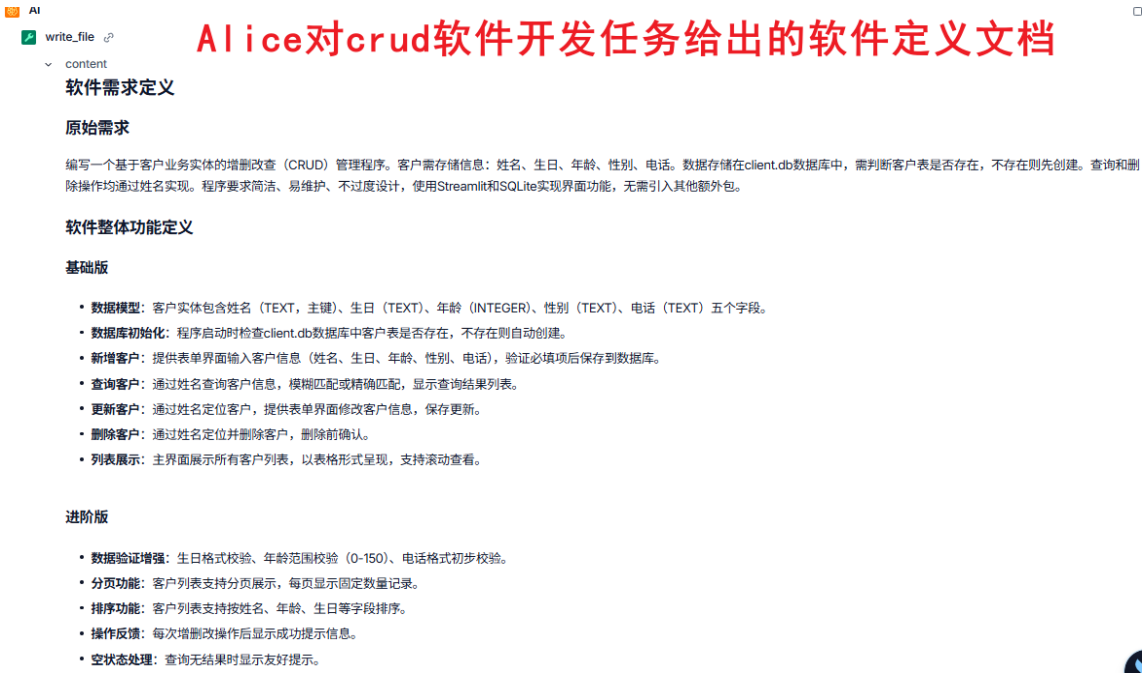


图 4.3 Planner Alice 对于 crud manage 软件开发任务给出的软件整体功能定义

在收到用户原始需求后，Planner Alice 进行需求分析，将软件成品按照“基础版”、“进阶版”、“顶级版”三个层次给出软件整体功能定义，其中基础版已经充分覆盖 crud manage 任务中的用户原始的软件功能需求，“进阶版”中则额外关注了数据验证增强、

分页功能、友好提示等增强用户体验的功能，如图 4.3 所示。之后，Alice 还将基于软件整体功能定义，给出三个层次的软件功能点定义如图 4.4 所示。这些软件功能点定义完全局限于“用户可感知”的 UI 层次的软件功能，完全不涉及技术细节，让用户能够明确把握自己到底需要增加那些软件功能。

在现实世界的软件开发中，特别强求快速交付，如果软件开发团队接手一个软件需求后，必须等到软件成品尽善尽美，那么软件客户很可能会等待得难以忍受。本文的 AgentTeam 在实际开发过程中，对于每个软件开发任务，要求 Planner Alice 明确指派 Builder Bob 先行完成基础版的所有软件功能点，待 Bob 完成后，必须向 Alice 请示下一步开发行动，而 Alice 将会向用户请示下一步开发需求，即用户是否要求继续“进阶版”“顶级版”范围内的软件功能点开发，在很大程度上允许用户看到最初可用的基础版软件系统，大大降低用户的苦等体验。



图 4.4　Alice 就 crud manage 软件开发任务给出的软件功能点定义

除了软件整体功能定义和软件功能点对于软件功能的良好划分，Builder Bob 和 Evaluator 的软件功能点验收标准协商和验收测试用例自测与复现，对确保最终软件成品的可用性起到了至关重要的作用。

软件功能点的粒度有粗有细，可以是最大的功能模块，或者最小的函数，其间又包含 UI 页面、UI 组件、后端接口、后端业务逻辑，因此软件功能点的验收标准不能按照统一的预定义模板进行规范，而是需要通过 Builder Bob 与 Evaluator Eva 通过协商确定。

如图 4.5 所示，Bob 提出的软件功能点验收标准提议中主要包含两部分，分别是当前软件功能点对应的“必要实现项”和验收测试用例。

值得注意的是，Bob 提出的每个验收测试用例必须是可执行、可客观验证的测试用例，大致分为 UI 交互操作测试用例与接口测试用例，前者确保了 UI 层的正确性，后者确保了软件代码逻辑的正确性。

仅仅依赖 Bob 完成软件功能点开发还有许多不足，比如软件代码结构问题，因为智能体对于自己的任务执行往往很难发现问题，因此需要有一个有效的评估者对其任务执行情况做出评价。



Task-03 验收标准提议

软件功能点'必要实现项'

1. 在UI中添加标题"新增客户"
2. 创建表单，包含以下输入字段：
   - 姓名：text_input，标签为"姓名"
   - 生日：text_input，标签为"生日（格式：YYYY-MM-DD）"
   - 年龄：number_input，标签为"年龄"，最小值为0
   - 性别：selectbox，标签为"性别"，选项为"男"、"女"
   - 电话：text_input，标签为"电话"
3. 表单包含"提交"按钮
4. 实现 `add_client()` 函数：
   - 连接数据库
   - 执行INSERT语句插入客户数据
   - 提交事务并关闭连接
5. 提交后显示成功提示信息

验收测试用例

- **用例1**：启动streamlit应用，在界面上能看到"新增客户"标题和包含姓名、生日、年龄、性别、电话的表单
- **用例2**：填写表单并提交，数据库中能查询到新插入的客户记录
- **用例3**：重复插入相同姓名客户，系统应提示错误（主键冲突）

请审核以上验收标准，如有修改意见请告知。

图 4.5　Bob 对当前进行的软件功能点提出验收标准提议

正如 Reflexion 模式中的二元奖励模型对每轮任务执行的最终结果做出成功或失败的判断，对于 Bob 的每个软件功能点实现，也需要 Evaluator 依据既定标准对其代码实现进行审阅，之后对测试用例进行复现。

如图 4.6 所示，对于 crud manage 这一软件开发任务的 TASK-03 这一软件功能点的验收测试用例 1 和验收测试用例 2（参见图 4.5），Builder Bob 在成功启动 streamlit 应用并调用 sandbox_page_use_extract 成功从当前 Web 页面抽取出页面内容后，Bob 判断验收测试用例 1 已经通过测试。

接着，Bob 连续调用多次 sandbox_page_use_act 进行表单输入操作，输入测试用户“张三”的姓名、生日、年龄、电话，期间调用 sandbox_page_use_extract 检验表单项是否成功接收输入，最终调用 sandbox_page_use_act 点击表单的提交按钮以验证当前软件功能点的验收测试用例 2。

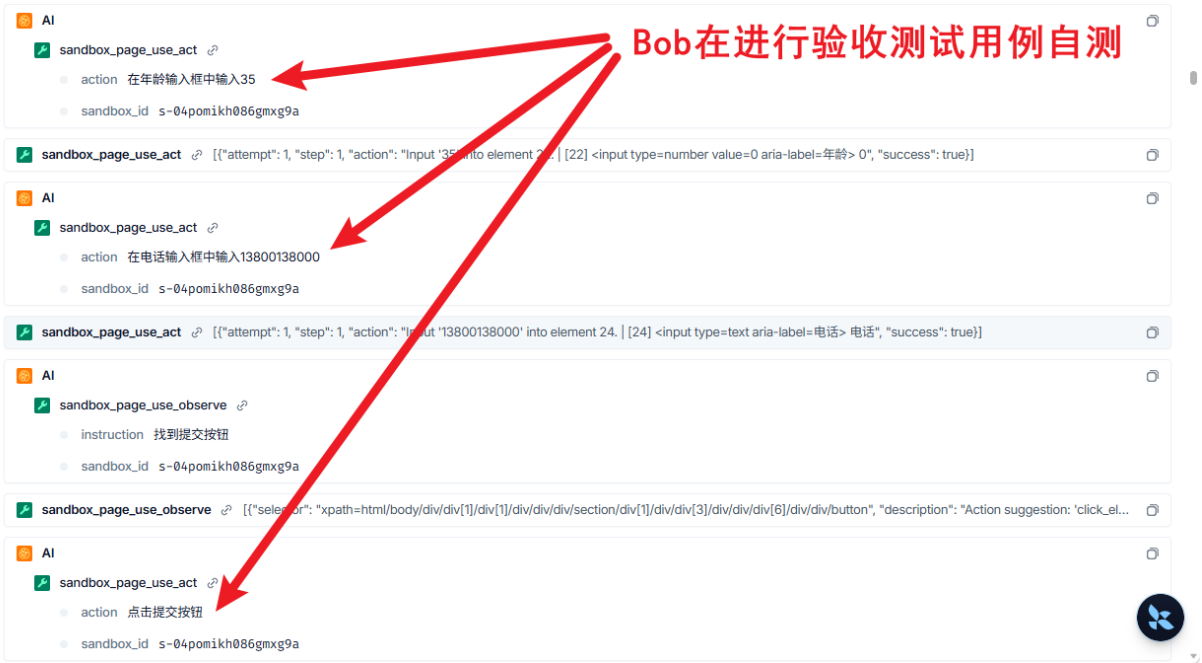


图 4.6 Builder Bob 对验收测试用例进行自测

如图 4.7 所示，对于 crud manage 的 TASK-03 软件功能点的验收测试用例 1，Bob 在执行 UI 交互操作后提交新客户插入后，一方面调用了 sandbox_page_use_extract 查看当前网页中是否出现“客户 张三 添加成功”的提示信息，另一方面同时调用了 execute 工具执行 Python 脚本检验 SQLite 数据库中是否确实有新插入的用户数据。工具调用的反馈明确告知 Bob 当前执行的测试用例符合预期结果。



图 4.7　Builder Bob 执行验收测试用例的结果

Evaluator Eva 作为软件质量评审专家，需要对 Bob 的每个软件功能点实现进行代码评审，重点评审每个软件功能点的代码实现是否遵循“UI、Runtime、Service、Repo、Config、Types”的 6 层分层结构，其中 UI 负责用户界面层的渲染展示、Runtiem 层负责用户交互事件的接收与分发，Service 层负责业务逻辑的处理，Repo 层负责数据的存储与访问，Config 层负责统一管理零碎的软件配置，Types 层负责类型定义。

一个优秀的软件功能点实现应该明确按照上述分层模式进行代码编写，同时严格遵循严格遵从上层只能依赖下层的规定（UI 为最上层，Types 为最下层）。除此之外，Eva 还负责对软件功能点的代码完成静态审阅，检查代码逻辑和代码命名规范、异常处理等问题。

Evaluator 最重要的一部分工作就是作为用户的代表，复现类别为“UI 交互操作验证”或“接口响应测试”的验收测试用例，确保软件产品每个功能点的可用性。如图 4.8 所示，Eva 同时执行 Python 脚本查询数据库信息，检查用户更新的软件功能点的实现的正确性，简单复现了 Bob 的自测结果，确保 Bob 实现的软件功能点的功能是正确可靠的。

Evaluator 和既往多智能体软件开发框架中的智能体角色，比如测试员、质量保证工

程师最大的区别在于，它并不是在在工程师给出整个软件的所有编码之后才开始对软件系统进行测试，之后再反馈测试意见给工程师要求其修复代码错误。Evaluator Eva 是在软件开发的过程中及时进行软件功能实现正确性测试任务，并对复现失败的测试用例进行反馈，如此增加了软件开发过程中过程性反馈，这是过往多智能体软件开发框架所缺失的。



图 4.8　Eva 对‘客户更新’软件功能点的测试用例进行复现

4.6　不足与展望

尽管 AgentTeam 在 SoftwareDev-7 数据集上取得了较好的软件成果物可用性表现，但是仍然存在着一些不足。

首先是统计数据表现存在优化空间。目前 AgentTeam 对于一个新的软件开发任务，即便在仅仅要求完成‘基础版’软件功能点的开发的条件下，仍然需要平均接近 70 分钟的等待时间。那么是否通过调整开发工作流是否能够在保障最终软件成品可用的条件下减少等待时间呢？通过考察具体的协作过程，可以发现很多时间耗费在执行或者复现

软件功能点验收测试用例上。由于目前的开发工作流要求对于每个软件功能点进行双重测试，即 Builder Bob 首先自行对软件功能点的实现执行验收测试用例，后续 Evaluator Eva 在评审软件功能点实现时仍然会复现（重新执行）验收测试用例，所以验收测试用例执行是提升软件生成耗时的重要突破口。观察发现，在纯前端应用开发任务下，Bob 和 Eva 在使用浏览器操作工具时常常遭遇无法访问此网站的错误，并在之后通过反复尝试才能正确启动前端页面，这说明 Bob 和 Eva 当前加载的 software-funtion-verification 技能的内容并不足以支持 Bob 和 Eva 熟练应对该类应用的功能验证，因此是否可以通过智能体自主优化已有技能的文件内容来提升后续的任务表现？智能体将工具使用经验加入到相应技能的文件内容中是否可以提升智能体在后续任务上的表现？这值得未来探索。

其次，Bob 和 Eva 双智能体协商得出的每个软件功能点的验收测试用例不保证能够充分验证软件功能点实现的正确性。对于某个软件功能点，如果协商结果中验收测试用例数量较少，可能并不足以充分验证实现代码的逻辑正确性，会出现验收测试用例执行的结果符合预期，但是最终软件成品可用性差劲的情况。是否需要新增验收测试用例的详细标注以最大限度规避其不完备性，或者在代码生成后立即要求 Eva 对代码逻辑执行静态评审以提前暴露问题？除此之外，是否应该新增某种类型的验收测试用例来解决，比如回归测试用例，每当新增一个软件功能点实现时，是否有必要执行回归测试用例以验证过去功能不被破坏？上述问题均有待探索。

最后，AgentTeam 对于软件开发方法学或者软件开发规范的遵守程度有限。观察软件成果物可以发现，目前软件成果物的项目结构中依然存在巨大的源码文件，随着开发过程推进，单个源码文件，比如单页 HTML 应用的代码行数不断增多，将不断增大后续的扩展难度，难以应对真实世界复杂应用的开发。是否可以通过既有开发框架默认的项目结构模板提升 AgentTeam 的开发规范性？是否应该新建智能体负责不断检查既有代码库以识别和偿还技术债务？这些问题也值得进一步的探索。

结论

本文针对基于大模型的软件自动开发场景中，现有多智能体框架存在的角色冗余、过程反馈缺失、代码可执行性差、运行环境不安全等核心问题，提出并实现了一套分层协作与平等对话融合、角色精简、反馈密集、环境隔离的多智能体协作方法，构建了轻量化软件开发专用多智能体团队 AgentTeam。该团队以“规划师-开发者-评估者”为核心角色，通过需求分级拆解、验收标准协商、自测与评审闭环、沙箱安全隔离四大机制，补齐了传统多智能体开发框架在功能可用性、过程可控性、运行安全性上的短板，显著提升了自动生成软件的实际可用率。

实验基于 SoftwareDev-7 数据集开展，以 ChatDev、MetaGPT 为基线，从开发耗时、词元消耗、代码规模、功能可用性等维度进行量化对比与人工验证。结果表明，本文方法在轻量化游戏、数据工具、管理系统等典型任务上，软件成果可用性达到 6/7，较 ChatDev 提升 42.8%，较 MetaGPT 提升 28.5%，在逻辑复杂度更高的游戏类项目中优势尤为突出。消融实验进一步证实，规划师的需求分层拆解与评估者的过程性评审是保障功能可用性的关键，二者共同将开发从“一次性代码生成”转变为“分步验证、迭代完善”的可控流程，从根源上降低了大模型幻觉导致的功能失效问题。

同时，本文为智能体配备浏览器自动化、桌面 UI 操控、文件系统与 Shell 执行等专业化工具，并通过技能系统注入工程最佳实践，使开发者智能体具备接近人类的工程化实现能力；基于 AgentBay 沙箱实现任务级环境隔离，解决了自动执行代码带来的本地安全风险，满足轻量化软件自动开发的安全刚需；基于 LangGraph 实现智能体独立状态持久化与棕地开发支持，提升了复杂开发任务的稳定性与连续性。

尽管当前方案在长耗时、高词元消耗上仍有优化空间，且在依赖复杂视觉感知的桌面端游戏开发中仍存在局限，但整体证明：面向轻量化软件的多智能体协作，可通过精简角色、强化过程反馈、工具赋能与安全隔离，显著超越现有框架的可用性表现。本文所提出的协作模式与工程实现，为大模型多智能体在自动化前端/全栈小项目、低代码工具生成、教学级软件开发等场景落地提供了可复用的技术路径，也为未来更高效、更经济、更通用的自动开发系统构建奠定了基础。

参考文献

[1] Tran K-T, Dao D, Nguyen M-D, et al. Multi-Agent Collaboration Mechanisms: A Survey of LLMs[J]. arXiv, 2025, 2501.06322.

[2] Zheng Q, Xia X, Zou X, et al. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X[C]. Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Long Beach, 2023: 5673-5684.

[3] Nijkamp E, Pang B, Hayashi H, et al. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis[C]. The Eleventh International Conference on Learning Representations, Kigali, 2023: 1-25.

[4] Fried D, Aghajanyan A, Lin J, et al. InCoder: A Generative Model for Code Infilling and Synthesis[C]. The Eleventh International Conference on Learning Representations, Kigali, 2023: 1-26.

[5] Dong Y, Jiang X, Liu H, et al. Generalization or Memorization: Data Contamination and Trustworthy Evaluation for Large Language Models[C]. Findings of the Association for Computational Linguistics, Bangkok, 2024: 12039-12050.

[6] Zhang K, Zhang H, Li G, et al. Toolcoder: Teach code generation models to use api search tools[J]. arXiv, 2023, 2305.04032.

[7] Zhang K, Li J, Li G, et al. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges[C]. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Bangkok, 2024: 13643-13658.

[8] Li J, Le H, Zhou Y, et al. Codetree: Agent-guided tree search for code generation with large language models[C]. Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Albuquerque, 2025: 3711-3726.

[9] Madaan A, Tandon N, Gupta P, et al. Self-refine: Iterative refinement with self-feedback[C]. Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, New Orleans, 2023: 46534-46594.

[10] Chen X, Lin M, Scharli N, et al. Teaching large language models to self-debug[C]. The Twelfth International Conference on Learning Representations, Vienna, 2024: 1-80.

[11] Olausson T X, Inala J P, Wang C, et al. Is self-repair a silver bullet for code generation?[C]. The Twelfth International Conference on Learning Representations, Vienna, 2024: 1-49.

[12] 张旭龙, 鲍习坤, 瞿晓阳, 等. 多代码智能体研究与应用综述[J]. 大数据, 2025: 1-21.

[13] 董益宏, 姜雪, 钱家如, 等. 基于大语言模型智能体的代码生成综述[J]. 软件学报, 2025: 1-27.

[14] Dong Y, Jiang X, Jin Z, et al. Self-Collaboration Code Generation via ChatGPT[J]. ACM Transactions on Software Engineering and Methodology, 2024, 33(7): 1-38.

[15] Lg G, Hammoud H, Itani H, et al. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society[C]. Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, New Orleans, 2023: 51991

-52008.
[16] Qian C, Liu W, Liu H, et al. ChatDev: Communicative Agents for Software Development[C]. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Bangkok, 2024: 15174-15186.
[17] Hong S, Zhuge M, Chen J, et al. Metagpt: Meta programming for multi-agent collaborative framework[C]. The Twelfth International Conference on Learning Representations, Vienna, 2024: 1-29.
[18] Chen W, Su Y, Zuo J, et al. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors[C]. The Twelfth International Conference on Learning Representations, Vienna, 2024: 1-43.
[19] Qian C, Xie Z, Wang Y, et al. Scaling large language model-based multi-agent collaboration [C]. The Thirteenth International Conference on Learning Representations, Singapore, 2025: 1-18.
[20] Hou X, Zhao Y, Liu Y, et al. Large Language Models for Software Engineering: A Systematic Literature Review[J]. ACM Transactions on Software Engineering and Methodology, 2024, 33 (8): 1-79.
[21] Wei J, Wang X, Schuurmans D, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models[C]. Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, New Orleans, 2022: 24824-24837.
[22] Yao S, Yu D, Zhao J, et al. Tree of thoughts: Deliberate problem solving with large language models[C]. Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, New Orleans, 2023: 11809-11822.
[23] Besta M, Blach N, Kubicek A, et al. Graph of thoughts: Solving elaborate problems with large language models[C]. Thirty-Eighth AAAI Conference on Artificial Intelligence, Vancouver, 2024: 17682-17690.
[24] Wang X, Wei J, Schuurmans D, et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models[C]. The Eleventh International Conference on Learning Representations, Kigali, 2023: 1-24.
[25] Xi Z, Chen W, Guo X, et al. The rise and potential of large language model based agents: A survey[J]. Science China Information Sciences, 2025, 68(2): 1-44.
[26] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models [C]. The Eleventh International Conference on Learning Representations, Kigali, 2023: 1-33.
[27] Shinn N, Cassano F, Gopinath A, et al. Reflexion: Language agents with verbal reinforcement learning[C]. Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, New Orleans, 2023: 8634-8652.
[28] Wang G, Xie Y, Jiang Y, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models[J]. Transactions on Machine Learning Research, 2024: 1-44.
[29] Schick T, Dwivedi-Yu J, Dessir R, et al. Toolformer: Language models can teach themselves to use tools[C]. Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, New Orleans, 2023: 68539-68551.
[30] Guo T, Chen X, Wang Y, et al. Large Language Model Based Multi-agents: A Survey of Pr

ogress and Challenges[C]. Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, Jeju, 2024: 8048-8057.

[31] LangChain overview[EB]. https://docs.langchain.com/oss/python/langchain/overview.

[32] Wang L, Ma C, Feng X, et al. A Survey on Large Language Model based Autonomous Agents[J]. Frontiers of Computer Science, 2024, 18(6): 1-26.

[33] Quick Start Examples | agentbay-ai/wuying-agentbay-sdk | DeepWiki[EB]. https://deepwiki.com/agentbay-ai/wuying-agentbay-sdk/2.2-quick-start-examples.

[34] Minimax M2.7 - 模型自我进化，以技术突破驱动生产力革新 | Minimax[EB]. https://www.minimaxi.com/models/text/m27.

附录 A Software-7 软件开发任务

序号	任务名称	需求描述
0	Snake game	Create a snake game.
1	Brick breaker game	Create a brick breaker game.
2	2048 game	Create a 2048 game for the web.
3	Flappy bird game	Write p5.js code for Flappy Bird where you control a yellow bird continuously flying between a series of green pipes. The bird flaps every time you left click the mouse. If it falls to the ground or hits a pipe, you lose. This game goes on indefinitely until you lose; you get points the further you go.
4	Tank battle game	Create a tank battle game.
5	Excel data process	Write an excel data processing program based on streamlit and pandas. The screen first shows an excel file upload button. After the excel file is uploaded, use pandas to display its data content. The program is required to be concise, easy to maintain, and not over-designed. It uses streamlit to process web screen displays, and pandas is sufficient to process excel reading and display. Please make sure others can execute directly without introducing additional packages.
6	CRUD manage	Write a management program based on the crud addition, deletion, modification and query processing of the customer business entity. The customer needs to save this information: name, birthday, age, sex, and phone. The data is stored in client.db, and there is a judgement whether the customer table exists. If it doesn't, it needs to be created first. Querying is done by name; same for deleting. The program is required to be concise, easy to maintain, and not over-designed. The screen is realized through streamlit and sqlite—no need to introduce other additional packages.

附录 B　软件成果物功能人工验证详情、AgentTeam 软件成果物

1　ChatDev 软件成果物功能人工验证详情

1.1　验证概述

本节报告 ChatDev 多智能体系统生成的 7 项软件成果物的人工验证结果，涵盖贪吃蛇游戏、打砖块游戏、2048 游戏、飞翔的小鸟游戏、坦克大战游戏、Excel 数据处理工具以及客户管理系统的功能完整性测试。验证过程在 Windows 操作系统环境下进行，采用 conda 管理 Python 依赖，确保各项目隔离运行。

1.2　贪吃蛇游戏（ChatDev-0）验证详情

功能	概述	验证结果
经典游戏玩法	传统蛇形游戏机制，搭配现代视觉效果	☑ 成功实现
分数系统	每吃一个食物+10 分	☑ 成功实现
暂停功能	随时暂停和恢复游戏	✗ 没有实现，不能随时暂停和恢复游戏
重新开始选项	游戏结束后快速重新开始	✗ 没有实现，按下 R 键不能重开游戏
碰撞检测	墙壁碰撞和自身碰撞检测	⚠ 成功实现墙体碰撞，但是蛇的移速过快，难以人工检验自身碰撞检测
视觉效果	网格背景、3D 风格蛇身和食物	☑ 成功实现

启动游戏后仅能完成单局游戏，并且蛇体移动速度过快，用户体验差，判定为不满足用户需求。

1.3　打砖块游戏（ChatDev-1）验证详情

功能	概述	验证结果
菜单系统	带有说明的启动画面	⚠ 成功实现，但是没有做到”按下任意键即可开始游戏“

分数追踪	每破坏一个砖块获得积分	❌ 不能开始游戏，无法验证
生命系统	每局 3 条命	❌ 不能开始游戏，无法验证
挡板物理	球的反弹角度根据击中挡板的位置变化	❌ 不能开始游戏，无法验证
多种游戏状态	菜单、游戏中、游戏结束、胜利	❌ 不能开始游戏，无法验证
彩色砖块	5 行不同颜色的砖块	❌ 不能开始游戏，无法验证
流畅游戏体验	60 FPS 渲染，响应式控制	❌ 不能开始游戏，无法验证

困死在游戏开始菜单，无法进入游戏，无法满足用户需求。

1.4 2048 游戏（ChatDev-2）验证详情

功能	描述	验证结果
游戏面板	4×4 网格，使用 CSS Grid 渲染，方块动态出现	❌ 浏览器报错，无法加载
分数和最高分数	实时更新当前分数，最高分数保存到浏览器 localStorage	❌ 浏览器报错，无法加载
控制方式	键盘方向键 (←↑→↓) 或屏幕上的 "New Game" / "Try Again" 按钮	❌ 浏览器报错，无法加载
游戏结束检测	无法移动时自动触发，提供重试按钮	❌ 浏览器报错，无法加载
响应式设计	可在桌面浏览器和大多数移动设备上运行	❌ 浏览器报错，无法加载
零安装游玩	服务器运行后，在任何现代浏览器中均可访问，无需额外插件	❌ 浏览器报错，无法加载

浏览器中打开 http://127.0.0.1:5000 后，显示 "TemplateNotFound jinja2.exceptions.TemplateNotFound: index.html"，显然，该产品不能满足用户需求。

1.5 飞翔的小鸟游戏（ChatDev-3）验证详情

功能	描述	验证结果
基于物理的飞行	重力拉小鸟下坠；鼠标点击给予向上的瞬间升力（拍打）	❌ 浏览器报错，无法加载

动态旋转	小鸟根据垂直速度平滑旋转，带来真实感	❌ 浏览器报错，无法加载
程序化管道	管道在随机高度生成，具有固定的垂直间隙	❌ 浏览器报错，无法加载
分数系统	每通过一对管道加 1 分；分数显示在左上角	❌ 浏览器报错，无法加载
渐进式难度	每 5 分管道速度增加且间隙变窄（速度上限为 5，最小间隙 100px	❌ 浏览器报错，无法加载
最高分持久化	最高分保存到浏览器的 localStorage，并在开始和游戏结束画面显示	❌ 浏览器报错，无法加载
状态机	三种游戏状态（START、PLAY、GAMEOVER），具有不同的 UI 覆盖层	❌ 浏览器报错，无法加载
暂停功能	按 P（或 p）暂停/恢复游戏	❌ 浏览器报错，无法加载
简单图形	使用内置 p5.js 形状（椭圆、三角形、矩形），无需外部图像资源	❌ 浏览器报错，无法加载

浏览器打开 http://localhost:8000 后一片灰暗，所有文件使用了 Python 风格的三引号注释 '"..."'，这在 HTML 和 JavaScript 中是无效的语法，显然，该软件成品不能满足用户需求。

1.6 坦克大战游戏（ChatDev-4）验证详情

功能	描述	验证结果
玩家移动	平滑的坦克移动，带有碰撞检测	☑ 成功实现
射击机制	基于坦克炮塔方向的射击	☑ 成功实现
敌人 AI	随机移动，与玩家对齐时智能射击	☑ 成功实现
生命值系统	所有坦克的可视化生命值条	☑ 成功实现
生命系统	玩家有 3 条生命，可重生	☑ 成功实现
分数系统	每消灭一个敌人得 10 分	☑ 成功实现
胜利条件	达到 20 个击杀获胜	☑ 成功实现
暂停功能	按 P 随时暂停/恢复	❌ 没有实现

核心功能全部实现，可认为满足了用户需求。

1.7 Excel 数据处理工具（ChatDev-5）验证详情

功能	描述	验证结果
Excel 文件上传	支持 .xlsx 和 .xls 文件格式	☑ 成功实现
数据预览	在交互式、可滚动的表格中显示 Excel 数据，占满容器宽度	☑ 成功实现
摘要统计	可选功能，查看数据的描述性统计	☑ 成功实现
错误处理	文件读取失败时显示优雅的错误消息	☑ 成功实现
简洁 UI	简单直观的界面，带有清晰的说明	☑ 成功实现

所有核心功能全部实现，可认为满足了用户需求。

1.8 客户管理系统（ChatDev-6）验证详情

功能	描述	验证结果
添加客户	填写表单并将新记录添加到数据库	☑ 全部实现
搜索/查看	按名称查找单个客户，或在表格中列出所有客户	☑ 全部实现
更新客户	定位现有记录（按 ID 或当前名称）并修改其任何字段	☑ 全部实现
删除客户	删除一条或多条记录（按名称或 ID）- 确认复选框可防止意外删除	☑ 全部实现

所有核心功能全部实现，可认为满足了用户需求。

2 MetaGPT 软件成果物功能人工验证详情

2.1 验证概述

本节报告 MetaGPT 多智能体系统生成的 7 项软件成果物的人工验证结果，涵盖贪

吃蛇游戏、打砖块游戏、2048 游戏、飞翔的小鸟游戏、坦克大战游戏、Excel 数据处理工具以及客户管理系统的功能完整性测试。验证过程在 Windows 操作系统环境下进行，部分项目在 60 分钟限时约束下进行了重试尝试。

2.2 贪吃蛇游戏（MetaGPT-0）验证详情

功能	描述	验证结果
游戏状态机	START → PLAYING → PAUSED → GAME_OVER → NAME_INPUT	❌ 无法验证，菜单卡死
难度选择	Easy (1) / Medium (2) / Hard (3)	❌ 无法验证，菜单卡死
蛇的移动与碰撞检测	墙壁碰撞和自身碰撞	❌ 无法验证，菜单卡死
食物系统	随机生成，不与蛇身重叠	❌ 无法验证，菜单卡死
分数系统	每吃一个食物 +10 分	❌ 无法验证，菜单卡死
排行榜	保存前 5 名最高分到 JSON 文件	❌ 无法验证，菜单卡死

开始菜单卡死，无法选择难度等级，进而无法开始游戏，不能满足用户需求。

2.3 打砖块游戏（MetaGPT-1）验证详情

功能	描述	验证结果
60fps 流畅游戏体验	-	☑ 成功实现
经典打砖块机制	挡板控制、球反弹、砖块破坏	⚠️ 部分实现（挡板控制成功，球太快无法验证）
游戏状态管理	开始、游戏中、游戏结束、胜利	⚠️ 部分实现（能开始但球速过快）
计分系统	击中砖块得分	❌ 无法验证
生命系统	3 条命	☑ 成功实现
视觉反馈	碰撞效果	❌ 无法验证

仅仅正确实现了从开始菜单进入游戏，游戏画面（挡板、砖块、生命）成功实现，但是刚从开始菜单进入游戏就有一颗球快速斜向下弹射，玩家根本来不及移动挡板反弹小球，玩家因此丧失一条命，再次之后玩家无法正常继续游戏，不符合用户需求。

2.4 2048 游戏（MetaGPT-2）验证详情

功能	描述	验证结果
键盘控制	使用 ↑ ↓ ← → 方向键滑动数字方块	☑ 成功实现
合并逻辑	相同数字合并，分数增加	☑ 成功实现
游戏状态	胜利条件（达到 2048）、游戏结束判定	☑ 成功实现
分数系统	实时显示当前分数和最高分	☑ 成功实现
新游戏按钮	随时重新开始	☑ 成功实现
本地存储	最高分持久保存	☑ 成功实现

所有核心功能正确实现，并且游戏画面相对美观，可认为满足了用户需求。

2.5 飞翔的小鸟游戏（MetaGPT-3）验证详情

功能	描述	验证结果
物理系统	重力和升力	☑ 成功实现
管道系统	随机生成，带缺口	☑ 成功实现
分数系统	通过管道时加分	☑ 成功实现
碰撞检测	与管道和地面的碰撞	☑ 成功实现
游戏状态机	START / PLAYING / GAME_OVER	☑ 成功实现

所有功能全部实现，满足用户需求。

2.6 坦克大战游戏（MetaGPT-4）验证详情

功能	描述	验证结果
玩家坦克控制	上下左右/WASD 移动	✗ 无法启动
射击机制	空格键发射子弹	✗ 无法启动
敌人 AI	随机移动和射击	✗ 无法启动
碰撞检测	墙壁、敌人、子弹碰撞	✗ 无法启动
分数和生命系统	-	✗ 无法启动

游戏状态	开始、游戏中、游戏结束、胜利	✗ 无法启动

游戏启动失败，报错 pygame.error: font not initialized。问题在于 UI 类在 pygame.init() 之前被实例化，导致字体初始化失败，不能满足用户需求。

2.7 Excel 数据处理工具（MetaGPT-5）验证详情

功能	描述	验证结果
文件上传	支持 .xlsx 和 .xls 格式	☑ 成功实现
数据读取	使用 pandas 读取 Excel 文件	☑ 成功实现
数据展示	以可滚动表格形式展示，占满容器宽度	☑ 成功实现
错误处理	优雅处理无效文件格式	☑ 成功实现

全部功能都成功实现，满足了用户需求。

2.8 客户管理系统（MetaGPT-6）验证详情

功能	描述	验证结果
添加客户	填写表单添加新客户	☑ 正确实现
列出/搜索	显示所有客户列表，支持按姓名搜索	☑ 正确实现
编辑客户	加载并更新现有客户信息	☑ 正确实现
删除客户	按姓名删除客户	☑ 正确实现

全部功能都成功实现，满足了用户需求。

3 AgentTeam 软件功能成果物功能人工验证详情

3.1 验证概述

本节报告 AgentTeam 多智能体系统生成的 7 项软件成果物的人工验证结果，涵盖贪吃蛇游戏、打砖块游戏、2048 游戏、飞翔的小鸟游戏、坦克大战游戏、Excel 数据处理工具以及客户管理系统的功能完整性测试。验证过程在 Windows 操作系统环境下进行，部分项目经历了多次迭代改进。总体而言，AgentTeam 系统生成的软件成果物质量显著优于基线系统，成功率达到 85.7%。

3.2 贪吃蛇游戏（AgentTeam-0）验证详情

功能	描述	验证结果
方向控制	方向键控制蛇的移动方向	☑ 成功实现
速度选择	支持慢/中/快三档速度选择	☑ 成功实现
蛇身移动	自动向前移动	☑ 成功实现
食物生成	随机生成红色食物（闪烁效果）	☑ 成功实现
吃食物逻辑	蛇头碰到食物，得分+10，蛇身+1	☑ 成功实现
障碍物系统	随机生成 5 个障碍物	☑ 成功实现
碰撞检测	撞墙/撞自己/撞障碍物时游戏结束	☑ 成功实现
分数显示	实时显示当前得分和最高分	☑ 成功实现
最高分记录	localStorage 存储历史最高分	☑ 成功实现
暂停/继续	空格键暂停/继续游戏	☑ 成功实现
重新开始	游戏结束后可重新开始	☑ 成功实现

没有发现明显问题，判定为满足了用户需求。

3.3 打砖块游戏（AgentTeam-1）验证详情

功能	描述	验证结果
挡板控制	鼠标/键盘控制挡板移动	☑ 成功实现
小球反弹	物理反弹系统	☑ 成功实现
砖块消除	小球击中砖块消除并得分	☑ 成功实现
计分系统	实时显示当前分数和最高分	☑ 成功实现
道具系统	加宽挡板、多球、慢速球等	✗ 未实现
连击系统	连续消除触发连击加分	✗ 未实现
多关卡递进	难度递增机制	✗ 未实现
暂停功能	P/Esc 暂停游戏	✗ 未实现
结束状态检测	游戏结束判定	☑ 成功实现
重新开始	游戏结束后重新开始	☑ 成功实现

基础版功能全部实现，满足了用户需求。

3.4 2048 游戏（AgentTeam-2）验证详情

功能	描述	验证结果
游戏初始化	4×4 网格生成，初始生成 2 个方块（2 或 4）	☑ 成功实现
方块移动	方向键控制所有方块移动	☑ 成功实现
方块合并	相同数字碰撞时合并，数值翻倍	☑ 成功实现
分数计算	合并时累计分数（合并值）	☑ 成功实现
新方块生成	每次有效移动后随机生成新方块	☑ 成功实现
游戏结束判定	网格填满且无法移动时结束	☑ 成功实现
新游戏功能	空格键或点击按钮重新开始	☑ 成功实现

基础版功能全部实现，满足了用户需求。

3.5 飞翔的小鸟游戏（AgentTeam-3）验证详情

功能	描述	验证结果
黄色小鸟渲染	屏幕显示小鸟角色	☑ 成功实现
重力下落	小鸟无操作时持续下落	☑ 成功实现
鼠标点击响应	点击左键小鸟扇动翅膀上升	☑ 成功实现
绿色管道生成	从右侧生成管道对，上下留间隙	☑ 成功实现
管道碰撞检测	小鸟撞管道时游戏结束	☑ 成功实现
地面碰撞检测	小鸟坠地时游戏结束	✗ 未实现
游戏结束界面	显示分数和重新开始提示	☑ 成功实现

基础版绝大部分功能已实现，满足了用户需求。

3.6 坦克大战游戏（AgentTeam-4）验证详情

功能	描述	验证结果
游戏主界面	显示游戏标题、开始游戏按钮、最高分数	☑ 成功实现

游戏主画面	包含砖墙、钢墙、玩家基地	☑ 成功实现
玩家坦克控制	WASD/方向键控制坦克四向移动	✗ 未实现 WASD 控制移动
玩家坦克射击	空格键发射子弹	✗ 未实现
敌方坦克	从地图顶部随机生成，自动移动射击	✗ 未实现，仅仅是固定于地图顶部
子弹系统	子弹碰墙体或坦克时销毁	✗ 未实现
碰撞检测	玩家与墙体、敌方坦克、子弹碰撞检测	☑ 成功实现
基地保护	基地被摧毁时游戏结束	✗ 未实现基地
计分系统	击毁敌方坦克获得分数	✗ 未实现
生命系统	玩家初始三条生命	☑ 成功实现
游戏结束界面	显示最终得分和重新开始按钮	☑ 成功实现
图层顺序	正确的渲染层级	✗ 异常

玩家坦克不可发射炮弹：按空格键无法发射子弹；图层顺序异常：渲染层级不正确。由于核心战斗功能（玩家射击）未实现且存在图层渲染问题，判定为不满足用户需求。

3.7 Excel 数据处理工具（AgentTeam-5）验证详情

功能	描述	验证结果
文件上传	支持上传 .xlsx 和 .xls 格式的 Excel 文件	☑ 成功实现
数据预览	在网页端直接查看 Excel 数据内容，以表格形式展示	☑ 成功实现
基本信息	显示数据的行数和列数统计信息	☑ 成功实现
错误处理	文件读取失败时显示友好的错误信息	☑ 成功实现

核心功能全部实现，满足了用户需求。

3.8 客户管理系统（AgentTeam-6）验证详情

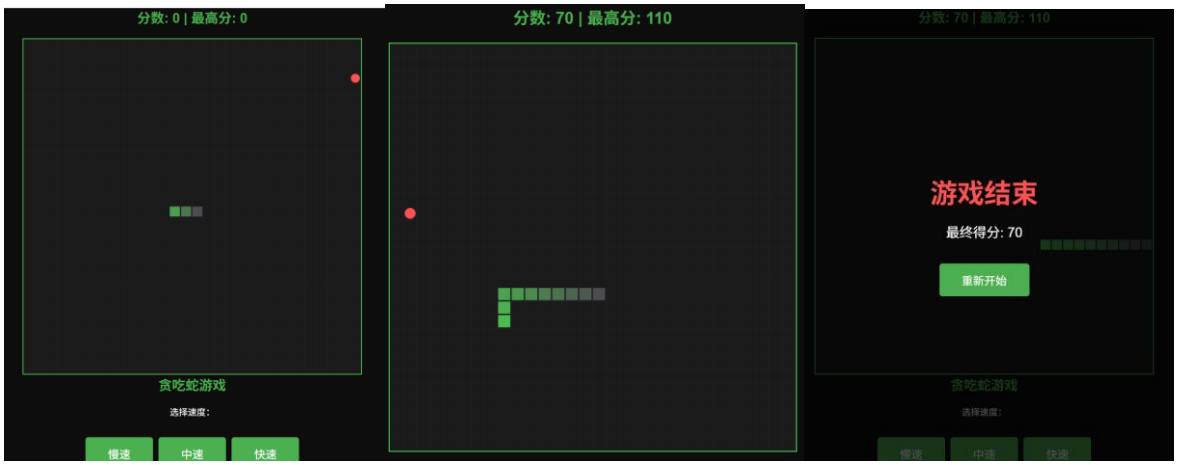
功能	描述	验证结果

新增客户	提供表单输入客户信息，验证必填项后保存到数据库	☑ 成功实现
查询客户	通过姓名精确查询客户信息，显示查询结果列表	☑ 成功实现
更新客户	通过姓名定位客户，修改信息并保存更新	☑ 成功实现
删除客户	通过姓名定位并删除客户，删除前需确认	☑ 成功实现
列表展示	主界面展示所有客户列表，以表格形式呈现	☑ 成功实现

核心功能全部实现，满足了用户需求。

4 AgentTeam 软件成果物

4.1 贪吃蛇游戏（AgentTeam-0）



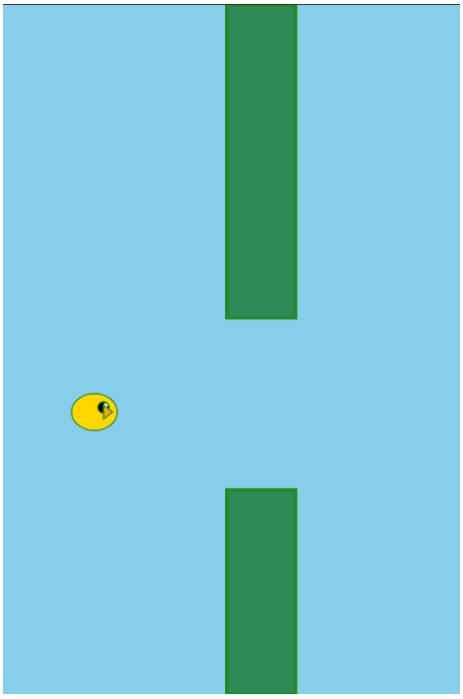
4.2 打砖块游戏（AgentTeam-1）



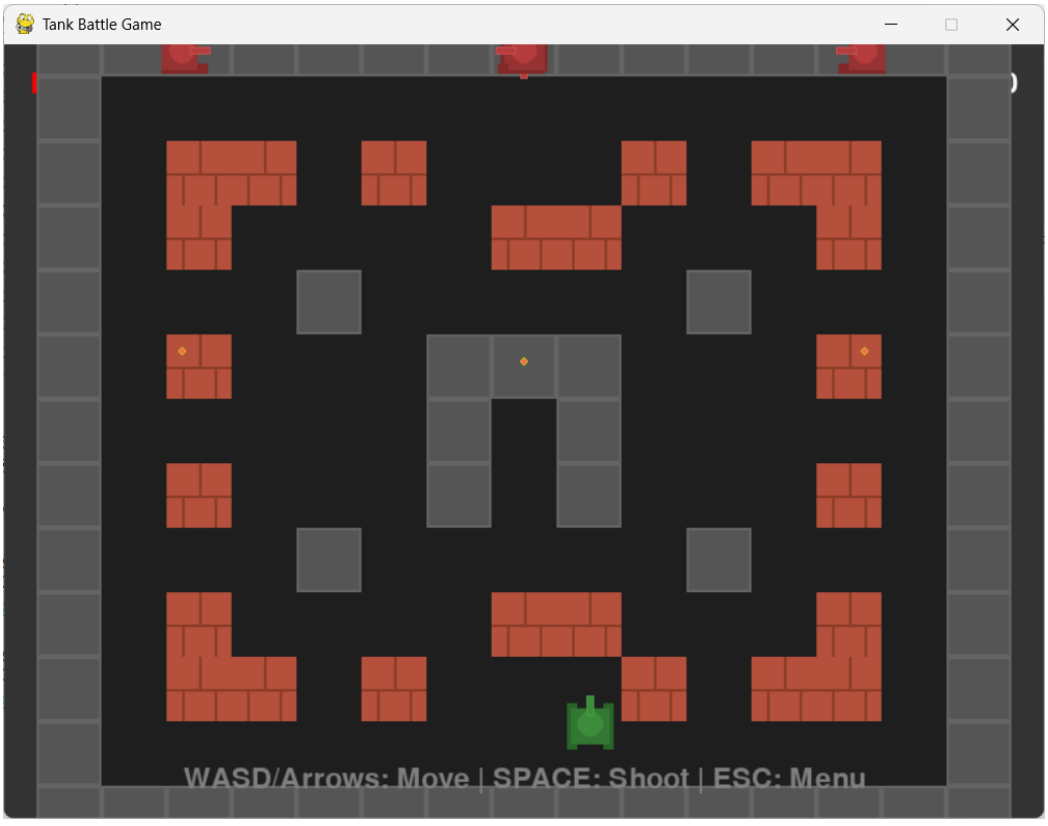
4.3 2048 游戏（AgentTeam-2）



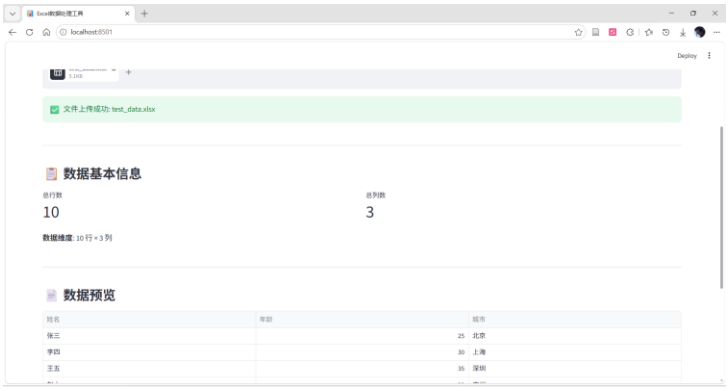
4.4 飞翔的小鸟游戏（AgentTeam-3）



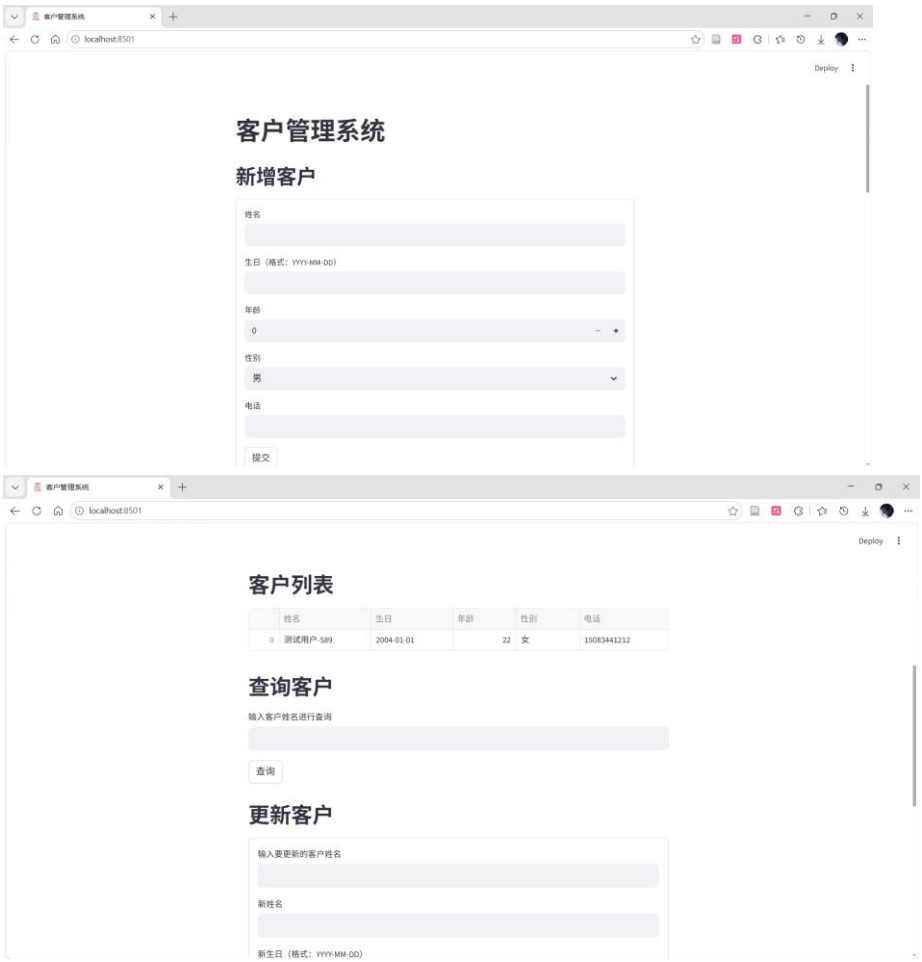
4.5 坦克大战游戏（AgentTeam-4）



4.6 Excel 数据处理工具（AgentTeam-5）



4.7 客户管理系统（AgentTeam-6）




客户管理系统
localhost:8501
Deploy
0
新性别
男
新电话
更新
删除客户
输入要删除的客户姓名
删除

修改记录

一、毕业论文（设计）题目修改

原题目：基于大语言模型的多智能体协作方法研究与实现

修稿后题目：基于大模型的软件开发多智能体协作方法研究

二、毕业论文（设计）内容重要修改记录

1. 封面学院、姓名等信息改成居中。

2. 更正了参考文献格式，补充了页码信息，补充了中文参考文献。

3. 修改第三章节结构，第四章新增新增角色消融实验。

4. 更正参考文献标题为宋体，将结论标题、附录标题更正为居中。

5. 统一图片大小，使图片内文字清晰；扩写了结论。

三、毕业论文（设计）外文翻译修改记录

1. 将外文翻译各节标题 2 个字符的缩进取消，保持与毕业论文格式一致。

2. 为外文翻译增加缺少的页眉。

四、毕业论文（设计）最后一次检测重复比

总文字复制比：1.5%，去除引用文献复制比：1.3%，去除本人文献复制比：1.5%，单篇最大文字复制比：0.8%。

记录人（签字）：

指导教师（签字）：

致谢

时光飞逝，转眼间大学生活即将画上一个圆满的句号。四年的学习生活，我在各方面都获得了成长与提升。

首先要诚挚感谢我的指导老师刘晗副教授，他渊博的专业知识、严谨的治学态度、平易近人的待人之道、积极乐观的生活态度，为我树立了榜样。在论文的选题、搜集资料和写作阶段都给予了我很大帮助，他们对我的关心和教诲我更将永远铭记。

感谢软件学院的全体教职工，在整个学习过程中，他们的支持和帮助是我顺利完成学业的重要保障。

感谢一直关心支持我的同学、学长学姐及朋友们，感谢你们的鼓励和帮助，愿我们友谊长存。

感谢我的母校——大连理工大学，大工为我提供了一个舒适宽广的学习平台， 让我不断汲取知识，充实自己。

感谢我的父母及家人，是他们在背后默默地支持我，给我无微不至的关怀，他们永远是我坚强的后盾。

最后，衷心感谢在百忙之中参与评阅论文和答辩评审的各位老师。